VOLUME II · THE WHOLE LANGUAGE

PYTHON, END TO END

The syntax first and completely. Then the same syntax at work. Then skeletons from one line to one package. Then the imports that matter, the annotated pieces you bolt together, and challenges that do not give you the answer.

Reference · Model · Skeletons · Pieces · Challenges · Appendix: the specimen

PYTHON 3.12+ | CLI & AUTOMATION | SYSADMIN | TEXT & LOGS | APIS | DATA | HINTS ONLY

HOW THIS VOLUME IS BUILT

Six movements, in the order you asked for them. Reference → model → skeletons → imports → pieces → challenges.

1. **Parts I–II** — every keyword, operator and builtin worth knowing, each with a one-line example. Then the execution model that makes them mean something.
2. **Parts III–V** — objects and types, laziness, and regular expressions properly.
3. **Part VI** — skeletons, from a one-liner to an installable package. Each is a template you will retype often.
4. **Part VII** — imports: standard library first, then the third-party libraries that have actually earned trust.
5. **Part VIII** — puzzle pieces. Working, annotated, built to be combined with the skeletons.
6. **Parts IX–X** — processes and concurrency, then challenges rated 1–5. Hints, never solutions.

Appendix A is the broken script: fifteen defects, no answer key in the main text. It ships as a real file next to this one.

• THE COLOUR CODE – LEARN IT ONCE

Pink is the rule you must not forget. **Cyan** is context. **Orange** is a trick or a memory hook. **Coral** is a trap. **Violet** is a worked example or an annotated piece. **Amber** is a challenge or a self-test – stop and do it.

HOW TO USE THIS

Reading code creates recognition. Writing, breaking and repairing code creates recall. For anything substantial:

1. **Predict** the output before running it.
2. **Type it** rather than pasting, at least once.
3. **Run it** with normal, empty, and hostile input.
4. **Break one thing deliberately** – a colon, a name, a type, a path.
5. **Read the traceback from the bottom up.**
6. **Repair it closed-book**, then compare.
7. **Vary it** – add a field, swap an implementation.
8. **Explain it aloud** in terms of objects, names, calls and returned values.

THE LABORATORY

```
mkdir -p ~/src/py-lab && cd ~/src/py-lab
python3 -m venv .venv
source .venv/bin/activate
python -VV
```

```
python3 -m py_compile script.py    # parse and compile, do not run
python3 -i script.py               # run, then drop into a REPL with its
names
python3 -m pdb script.py           # run under the debugger
python3 -m unittest -v             # standard-library tests
python3 -c 'import x; print(dir(x))'   # interrogate anything
```

• CONTEXT

`-m` means “find this module and run it as a program.” It is the reliable way to run a tool with a *particular* interpreter, which matters the moment you have more than one Python on the machine.

CONTENTS

PART I – THE REFERENCE

1 Keywords, all of them
2 Soft keywords and the underscore
3 Operators, by precedence
4 Builtins worth knowing cold
5 Literals and how to write them
6 The f-string mini-language
7 Punctuation map

PART II – SYNTAX IN USE

8 Names, objects, and rebinding
9 Expressions, statements, calls
10 Truth, absence, and boundaries
11 Control flow
12 match, properly
13 Comprehensions and when to stop
14 Functions are contracts
15 Scope, closures, LEGB
16 Exceptions as typed failure paths

PART III – OBJECTS AND TYPES

17 The core types
18 Dataclasses
19 Classes, and when not to
20 __slots__, descriptors, metaclasses
21 Type hints in real depth
22 Protocols over ancestry
23 Decorators and context managers

PART IV – LAZINESS

24 Iterables, iterators, generators
25 Generators beyond the basics
26 itertools as a vocabulary

PART V – REGULAR EXPRESSIONS

27 The syntax, completely
28 The re API
29 Writing regexes you can still read

PART VI – SKELETONS, SMALL TO LARGE

30 The one-file script
31 The function-shaped script
32 The argparse CLI
33 The tested module
34 The installable package
35 The long-running service

PART VII – IMPORTS

36 What each form binds
37 Modules, packages, and caching
38 The standard library you will actually use
39 Vetted third party, and how to judge one

PART VIII – PUZZLE PIECES

40 Files and directories
41 Text and log processing
42 Sysadmin and the environment
43 HTTP: calling an API
44 HTTP: serving an API
45 Data and charts
46 Orientation: desktop GUI
47 Orientation: machine learning

PART IX – PROCESSES AND CONCURRENCY

48 subprocess, correctly
49 Threads for I/O
50 Processes for CPU

PART X – CHALLENGES

51 How the challenges work

52 Level 1–2: reproduce and isolate

53 Level 3: build something whole

54 Level 4: break it on purpose

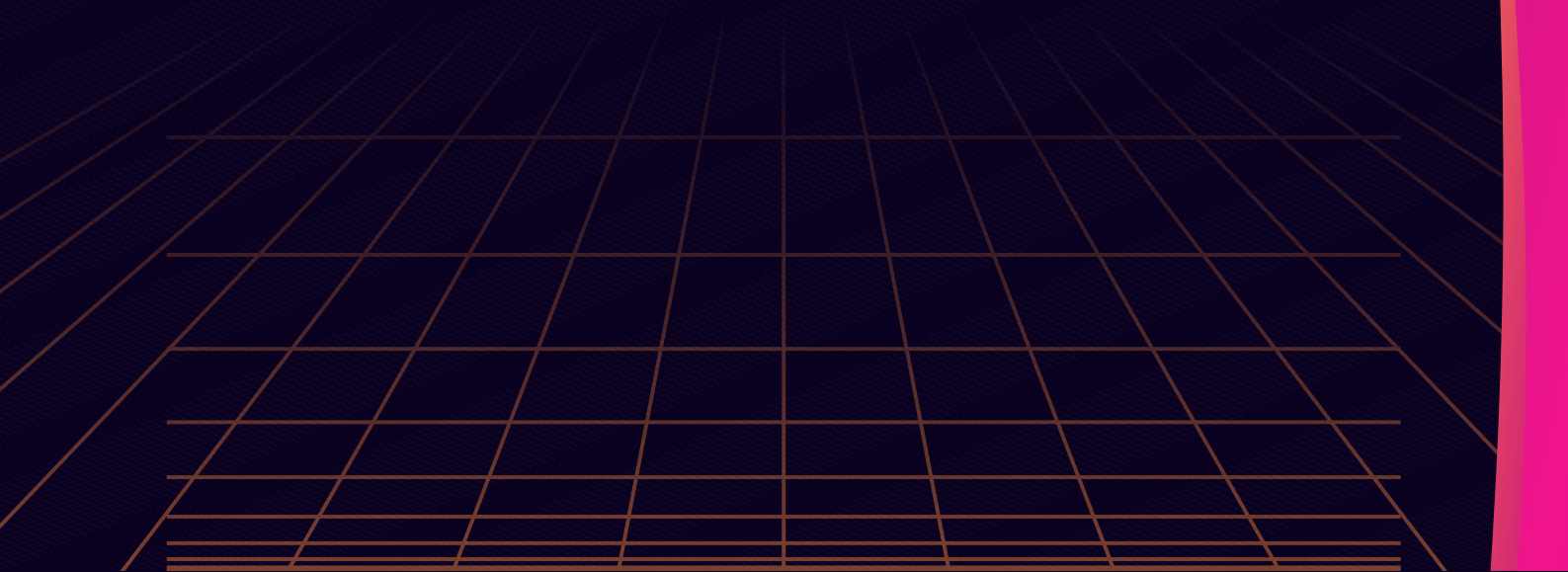
55 Level 5: redesign under constraint

APPENDICES

56 Appendix A – the specimen, dissected

57 Appendix B – errors you will see

58 Appendix C – what competence looks like



PART I

I

THE REFERENCE

Everything the language reserves, everything it operates with, and everything it hands you for free. One line of example each. Read it once end to end; come back to it constantly.

1 KEYWORDS, ALL OF THEM

Thirty-five reserved words. You cannot use any of them as a name. `python3 -c 'import keyword; print(keyword.kwlist)'` prints the list your interpreter actually enforces.

1.1 Values and identity

KEYWORD	MEANS	ONE LINE
`True`	Boolean true; an `int` subclass equal to 1	`total = True + True # 2`
`False`	Boolean false; equal to 0	`sum([True, False, True]) # 2`
`None`	The absence of a value; a singleton	`if result is None: ...`
`is`	Same object, not equal value	`a is b # identity`
`not`	Boolean negation	`if not names: return`
`and`	Short-circuit; returns an operand	`name = user and user.name`
`or`	Short-circuit; returns an operand	`port = given or 8080`
`in`	Membership; also the loop separator	`if ".py" in path.suffix: ...`

1.2 Control flow

KEYWORD	MEANS	ONE LINE
`if`	Conditional branch	`if size > 0: keep(path)`
`elif`	Chained alternative	`elif path.is_dir(): walk(path)`
`else`	Fallback – on `if`, loops, and `try`	`else: report(path)`
`for`	Iterate an iterable	`for line in handle: ...`
`while`	Repeat while true	`while attempts: attempts -= 1`
`break`	Leave the innermost loop	`if found: break`
`continue`	Next iteration immediately	`if line.startswith("#"): continue`
`pass`	Syntactically required nothing	`except KeyError: pass`

> • **TRAP – THE LOOP `ELSE`**
>
> `else` on a `for` or `while` runs when the loop finished **without** `break`. It is not "otherwise." Read it as `nobreak`.

1.3 Functions and returning

KEYWORD	MEANS	ONE LINE
`def`	Bind a function object to a name	`def size(p): return p.stat().st_size`
`return`	Hand a value back; ends the call	`return 0`
`yield`	Produce a value, suspend, resume later	`yield path`
`lambda`	A single-expression anonymous function	`sorted(rows, key=lambda r: r.size)`
`global`	Assign to a module-level name	`global COUNT`
`nonlocal`	Assign to an enclosing function's name	`nonlocal seen`

1.4 Classes, context, exceptions

KEYWORD	MEANS	ONE LINE
`class`	Build a class object, bind it to a name	`class Record: ...`
`with`	Acquire and guarantee release	`with p.open() as fh: data = fh.read()`
`as`	Bind a name – in `with`, `except`, `import`	`import numpy as np`
`try`	Guard a block that may raise	`try: risky()`
`except`	Handle one exception type	`except PermissionError as e: warn(e)`
`finally`	Always run, raise or not	`finally: handle.close()`
`raise`	Signal a failure	`raise ValueError("size must be >= 0")`
`assert`	State an internal invariant	`assert size >= 0, size`
`del`	Unbind a name or remove an item	`del cache[key]`

1.5 Modules and the async pair

KEYWORD	MEANS	ONE LINE
`import`	Find, execute once, cache, bind	`import json`
`from`	Select what an import binds	`from pathlib import Path`
`async`	Declare a coroutine function	`async def fetch(url): ...`
`await`	Suspend until a coroutine resolves	`body = await response.text()`

• ON ASYNC

Listed for completeness because they are reserved words – you cannot name a variable `async`. This volume does not teach the event loop. When you meet it, the mental model you need is Part IV: a coroutine is a suspendable call, the same way a generator is.

2 SOFT KEYWORDS AND THE UNDERSCORE

Four words are keywords only in context. Elsewhere they are ordinary names, which is why `match = re.match(...)` still works.

WORD	KEYWORD WHEN	ONE LINE
`match`	Heading a `match` statement	`match command.split():`
`case`	Inside a `match` block	`case ["get", key]:`
`type`	Declaring a type alias (3.12+)	`type Rows = list[dict[str, str]]`
`_`	The wildcard pattern in `case`	`case _: raise ValueError(command)`

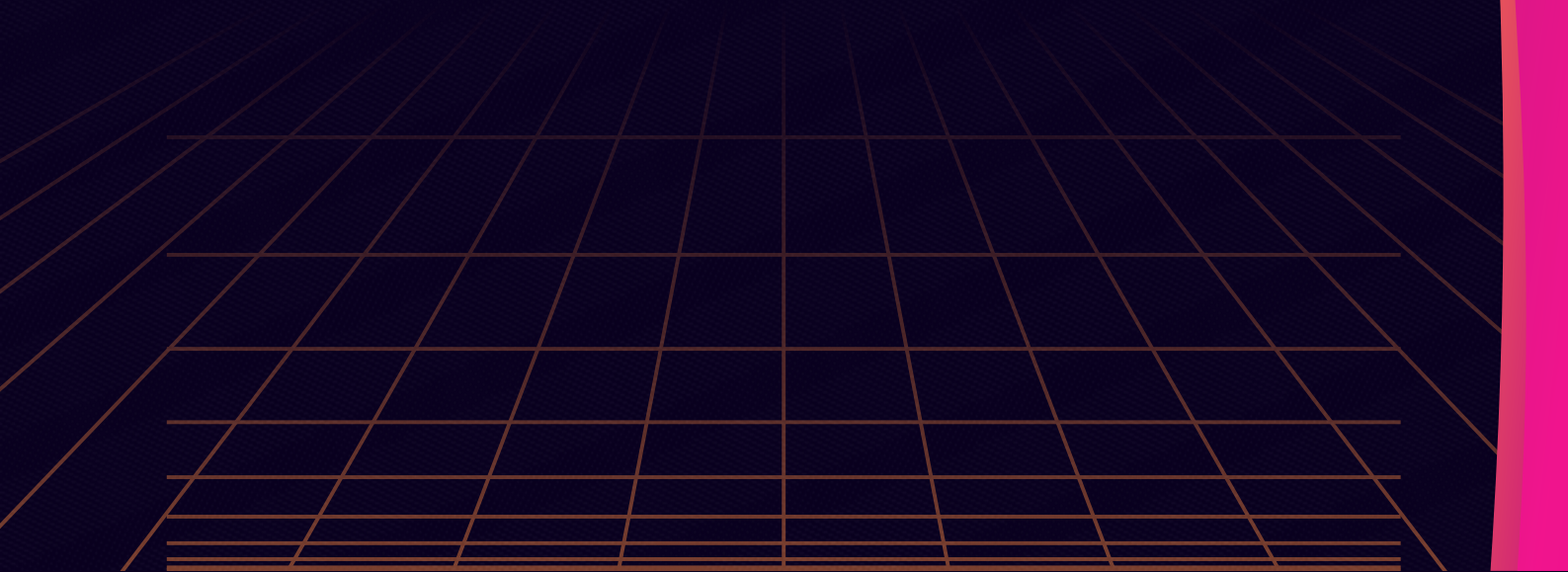
THE FOUR LIVES OF THE UNDERSCORE

- `_` – "I must bind this but will not use it": `for _ in range(3):`
- `_` – the last expression's value, in the interactive REPL only.
- `_name` – convention: internal, not part of the public API.
- `__name` inside a class – name mangling to `_Class__name`. Rare, and usually not what you want.

Also: `1_000_000` is a legal integer. Underscores in numeric literals are pure punctuation.

3 OPERATORS, BY PRECEDENCE

Highest binding first. When in doubt, parenthesise – the reader you are helping is yourself in three months.



OPERATOR	DOES	ONE LINE
`()` `[]` `.`	Call, subscript, attribute	`rows[0].name.strip()`
`**`	Power (binds right)	`2 ** 10 # 1024`
`+x` `-x` `~x`	Unary plus, minus, bitwise not	`-abs(delta)`
`*` `/` `//` `%` `@`	Multiply, true divide, floor divide, modulo, matrix multiply	`7 // 2 # 3`
`+` `-`	Add, subtract – also concatenate	`"a" + "b"`
`<<` `>>`	Bit shift	`1 << 8 # 256`
`&` `^` `|`	Bitwise and, xor, or – also set ops	`{1,2} & {2,3} # {2}`
`== != < <= > >=`	Comparison, chainable	`0 <= index < len(rows)`
`is` `is not`	Identity	`value is not None`
`in` `not in`	Membership	`key not in cache`
`not` `and` `or`	Boolean, short-circuiting	`a and not b`
`x if c else y`	Conditional expression	`root = Path(arg) if arg else Path.cwd()`
`:=`	Assign inside an expression	`while (line := fh.readline()): ...`
`|` in annotations	Union type	`def f(x: int | None) -> str: ...`
`/` in `dict` and `Path`	Overloaded: merge, join	`Path("/tmp") / "log.txt"`
`*` `**` at call sites	Unpack a sequence / a mapping	`func(*args, **kwargs)`

• TRAP — CHAINED COMPARISON

`0 <= i < n` is genuinely two comparisons with `i` evaluated once. But `a == b == c` means `a == b and b == c`, which is almost never what someone writing it expects in a filter.

• TRAP — `/` VERSUS `//`

`7 / 2` is `3.5`, a float, always. `7 // 2` is `3`. And `-7 // 2` is `-4`, not `-3` — floor division rounds toward negative infinity.

4 BUILTINS WORTH KNOWING COLD

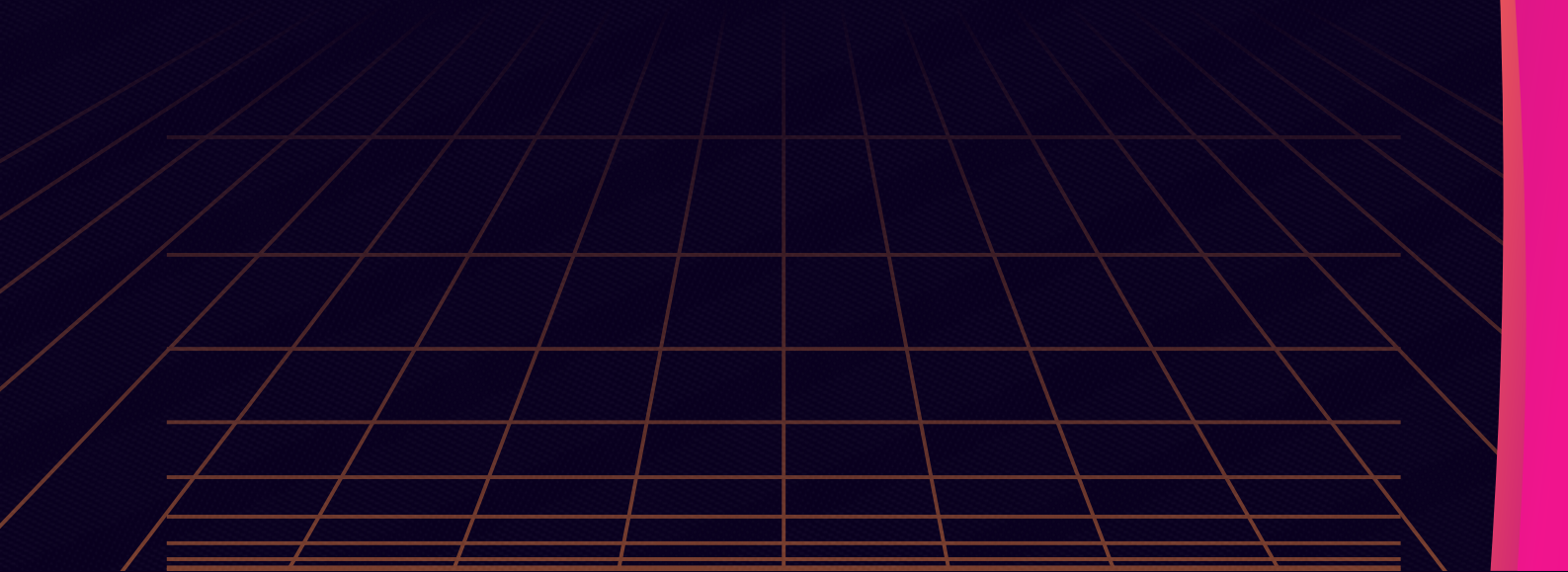
Available with no import at all. There are about seventy; these are the ones that earn their place in working memory.

4.1 Construct and convert

BUILTIN	DOES	ONE LINE
`int()`	To integer; base optional	`int("ff", 16) # 255`
`float()`	To float	`float("3.14")`
`str()`	Human-readable text	`str(Path("/tmp"))`
`repr()`	Unambiguous text, for debugging	`print(repr(value))`
`bool()`	Truthiness as a value	`bool([]) # False`
`bytes()`	Immutable byte string	`bytes("hi", "utf-8")`
`list() tuple() set() dict()`	Build a container	`dict(zip(keys, values))`
`frozenset()`	Hashable set — usable as a dict key	`frozenset({"a","b"})`

4.2 Inspect

BUILTIN	DOES	ONE LINE
`type()`	The object's class	`type(path)`
`isinstance()`	Type test, honours subclasses	`isinstance(x, (int, float))`
`dir()`	Names on an object	`dir(datetime)`
`help()`	The docstring, interactively	`help(str.partition)`
`vars()`	`__dict__` of an object	`vars(args)`
`id()`	Identity as an integer	`id(a) == id(b)`
`callable()`	Can it be called	`callable(handler)`
`hasattr() getattr() setattr()`	Attributes by name	`getattr(cfg, "port", 8080)`



4.3 Iterate and aggregate

BUILTIN	DOES	ONE LINE
`len()`	Item count	`len(rows)`
`range()`	Lazy arithmetic sequence	`for i in range(0, 10, 2):`
`enumerate()`	Index plus item	`for n, line in enumerate(fh, 1):`
`zip()`	Walk sequences in lockstep	`zip(names, sizes, strict=True)`
`sorted()`	New sorted list	`sorted(rows, key=len, reverse=True)`
`reversed()`	Lazy reverse iterator	`for x in reversed(items):`
`sum() min() max()`	Aggregate	`max(sizes, default=0)`
`any() all()`	Short-circuiting quantifiers	`any(p.suffix == ".py" for p in paths)`
`iter() next()`	Get an iterator; pull one item	`first = next(it, None)`
`map() filter()`	Lazy transform / select	`map(str.strip, lines)`

• **TWO ARGUMENTS THAT PREVENT REAL BUGS**

`zip(a, b, strict=True)` raises if the lengths differ instead of silently truncating to the shorter. `max(values, default=0)` returns the default instead of raising on an empty sequence. Both turn a silent logic failure into a loud one.

4.4 Input, output, and the rest

BUILTIN	DOES	ONE LINE
`print()`	Write to a stream; returns `None`	`print(msg, file=sys.stderr)`
`input()`	Read one line from stdin	`name = input("name: ")`
`open()`	A file object – use with `with`	`with open(p, encoding="utf-8") as fh:`
`format()`	Apply a format spec	`format(0.5, ".1%") # '50.0%'`
`round() abs() divmod()`	Numeric utilities	`divmod(17, 5) # (3, 2)`
`breakpoint()`	Drop into the debugger here	`breakpoint()`

5 LITERALS AND HOW TO WRITE THEM

```
42          1_000_000     0xFF      0o755      0b1010     # int
3.14        1e-9          2.5e3                           # float
3 + 4j                                                    # complex
"text"      'text'        """spans
lines"""                                                  # str
b"\x00\xff"                                               # bytes
r"C:\logs\new"                                            # raw: no
escape processing
f"{name!r} is {size:,} bytes"                             # f-string
rb"\d+"                                                   # raw bytes,
common in regex
"auto" "concat"                                           # adjacent
literals join
[1, 2]  (1, 2)  {1, 2}  {"k": 1}  (1,)  ()  []  {}  set()  # containers
```

• TRAP – THE LONELY TUPLE

`(1)` is the integer 1. `(1,)` is a one-item tuple. The comma makes the tuple, not the parentheses. And `{}` is an empty dict – an empty set is only ever `set()`.

• ALWAYS RAW FOR REGEX

Write every pattern as `r"..."`. Without it, `"\d"` is a string containing a backslash and a `d` only by accident of Python's escape rules – and `"\b"` silently becomes a backspace character instead of a word boundary.

6 THE F-STRING MINI-LANGUAGE

Format specs are their own small language and they pay for themselves in report output alone.

```
name, size, ratio = "app.log", 1048576, 0.4237

f"{name}"              # app.log
f"{name!r}"            # 'app.log'          !r repr, !s str, !a ascii
f"{name:>20}"          # right-align in 20 columns
f"{name:<20}"          # left-align
f"{name:^20}"          # centre
f"{name:.<20}"         # pad with dots instead of spaces
f"{size:,}"            # 1,048,576
f"{size:_}"            # 1_048_576
f"{size:>12,}"         # aligned and grouped
f"{ratio:.1%}"         # 42.4%
f"{ratio:.3f}"         # 0.424
f"{size:.2e}"          # 1.05e+06
f"{size:x}"            # 100000     hex; X upper, o octal, b binary
f"{size:#x}"           # 0x100000   with the prefix
f"{size=}"             # size=1048576          self-documenting
f"{ratio=:.2f}"        # ratio=0.42
```

• PRINTING A LITERAL BRACE

A single brace opens an interpolation, so to emit one you double it: two opening braces in the source produce one opening brace in the output, and the same for closing. That is the only escape the mini-language has. It is why a JSON template or a CSS rule written inside an f-string needs every brace doubled – and why, past one or two, you should build the string with `json.dumps` or plain concatenation instead.

• WORKED EXAMPLE – AN ALIGNED REPORT LINE

```
f"{size:>12,}  {stamp:%Y-%m-%d %H:%M}  {path}"
```

Twelve columns, comma-grouped, then a `datetime` formatted inline with `strftime` codes, then the path. A `datetime` understands format specs directly – you never need `.strftime()` inside an f-string.

7 PUNCTUATION MAP

```
function(arg)              # call
sequence[index]            # subscription
mapping[key]               # key lookup
object.attribute           # attribute access
name = value               # binding
name: Type = value         # annotated binding
parameter: Type            # parameter annotation
def f() -> Type:           # return annotation
@decorator                 # decorator: f = decorator(f)
*args  **kwargs            # collect / unpack
x, y = pair                # tuple unpacking
first, *rest = items       # starred unpacking
{"key": value}             # dict
{value1, value2}           # set
[expr for x in xs]         # list comprehension
{k: v for k, v in pairs}   # dict comprehension
(x for x in xs)            # generator expression
a[1:5:2]                   # slice: start, stop, step
a[::-1]                    # reversed copy
Path(root) / filename      # Path overloads / to join
f"{value}"                 # interpolation
lambda x: x.size           # anonymous function
:=                         # assignment expression
...                        # Ellipsis: a real object, often a stub body
```

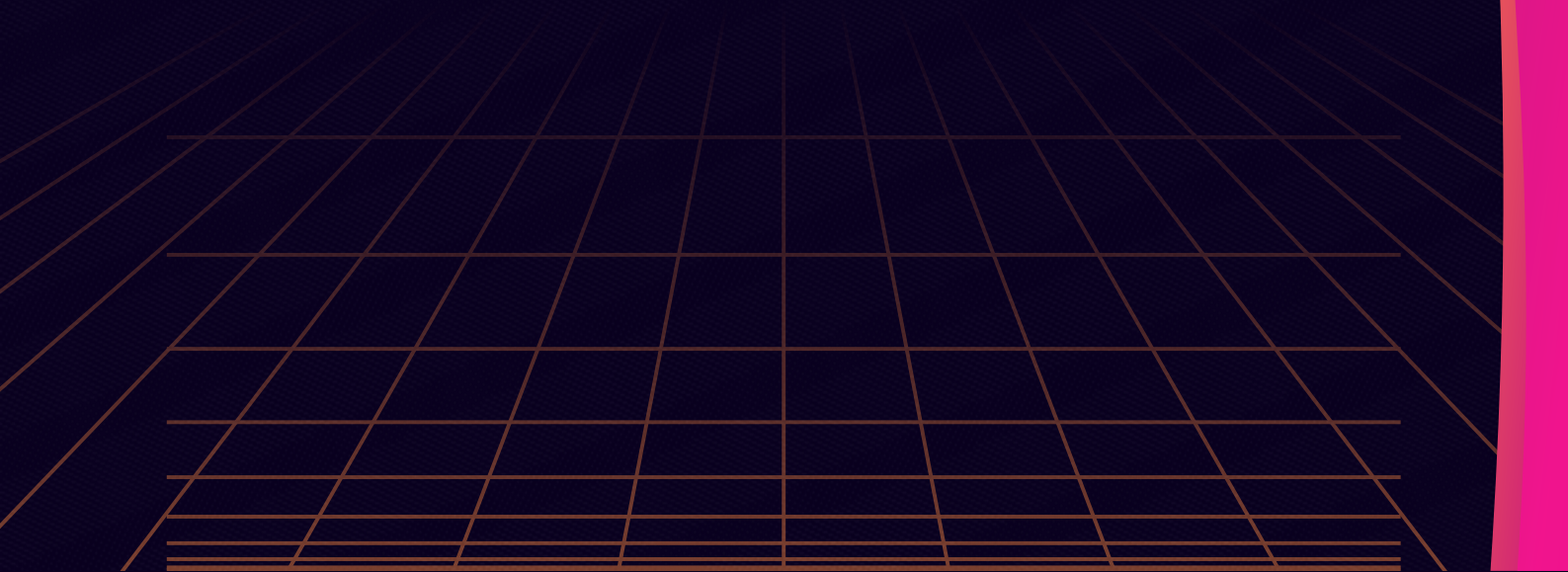
• CHALLENGE LEVEL 1

Without running anything, give the value and the type of each: `7 // -2` · `(1)` · `(1,)` · `{}` · `bool("False")` · `True + True` · `"a" * 3` · `[0] * 3`.

Hint: two of these surprise almost everyone, and one of them is about what makes a tuple.

PROGRESS — PART I

- [ ] Read all 35 keywords once, end to end
- [ ] Can explain the loop `else` without saying “otherwise”
- [ ] Know why `-7 // 2` is `-4`
- [ ] Used `zip(strict=True)` and `max(default=)` at least once
- [ ] Wrote an aligned, comma-grouped report line from memory
- [ ] Completed the Level 1 challenge closed-book



PART II

II

SYNTAX IN USE

The reference told you what the symbols are. This part tells you what the interpreter does when it meets them — which is the only thing that turns memorised syntax into predictions you can trust.

8 NAMES, OBJECTS, AND REBINDING

• THE LOAD-BEARING SENTENCE

Variables are names bound to objects. They are not shell-style textual substitution, and not statically typed boxes that hold a value.

```
path = Path("/tmp/report.txt")
alias = path
path = Path("/tmp/other.txt")
```

After line two both names point at the same object. Line three *rebinds* `path`; `alias` is untouched. Now contrast mutation:

```
names = ["a", "b"]
alias = names
names.append("c")
print(alias)          # ['a', 'b', 'c']
```

The list itself changed, so both names observe it.

• MEMORY HOOK

Rebind moves the label. Mutate changes the thing. Ask which one a line does and the surprise disappears — in function arguments, in class attributes, in copied collections, in defaults.

8.1 Copies are shallower than you think

```
import copy

grid = [[0, 0], [0, 0]]
shallow = grid.copy()       # or list(grid), or grid[:]
shallow[0][0] = 9
print(grid)                 # [[9, 0], [0, 0]]   the inner lists are
shared

deep = copy.deepcopy(grid)  # independent all the way down
```

8.2 Equality and identity

```
a, b = [1, 2], [1, 2]
a == b      # True   same values
a is b      # False  different objects
```

• TRAP

Use `is` only for sentinels – `None`, `True`, `False`, and your own module-level markers. `value is 3` and `name is "root"` appear to work for small ints and short strings because of interning, then fail on real data.

9 EXPRESSIONS, STATEMENTS, CALLS

An **expression** produces a value. A **statement** performs a language-level action. `path.stat().st_mtime` is five operations, left to right:

1. Resolve the object bound to `path`.
2. Retrieve its `stat` attribute – a bound method object.
3. Call it with no arguments.
4. Receive an `os.stat_result`.
5. Retrieve that result's `st_mtime` attribute.

• KEY IDEA

Parentheses call. A dot looks up an attribute. Brackets subscript. They combine freely, but each symbol keeps its own job – there is no context in which `()` indexes.

Interrogate rather than guess. This is the single highest-value habit in the book:

```
from pathlib import Path
p = Path(".")
print(type(p), type(p.stat), type(p.stat()))
print([n for n in dir(p) if not n.startswith("_")])
```

10 TRUTH, ABSENCE, AND BOUNDARIES

Falsy: `False None 0 0.0 "" [] {} set() ()`. Everything else is truthy, including `"0"`, `"False"`, and `[0]`.

```
if not filenames:          # empty and absent mean the same here
    continue

if limit is None:          # no limit supplied
    ...
elif limit == 0:           # an explicit zero, which is different
    ...
```

• THE BUG THIS CAUSES, CONSTANTLY

```
port = given or 8080       # given=0 silently becomes 8080
port = 8080 if given is None else given   # correct
```

Empty is not absent. `0`, `""` and `[]` are answers; `None` is the absence of one.

11 CONTROL FLOW

```
if path.is_file():
    inspect_file(path)
elif path.is_dir():
    inspect_directory(path)
else:
    report_special(path)
```

• KEY IDEA

Indentation is syntax. Four spaces. Never tabs, never mixed – that is what `TabError` is for.

```
for path in paths:
    if path.name.startswith("."):
        continue
    if should_stop(path):
        break
    inspect(path)
else:
    print("completed without break")
```

```
attempts = 3
while attempts > 0:
    attempts -= 1
    if probe():
        break
```

11.1 Bash instincts to revise

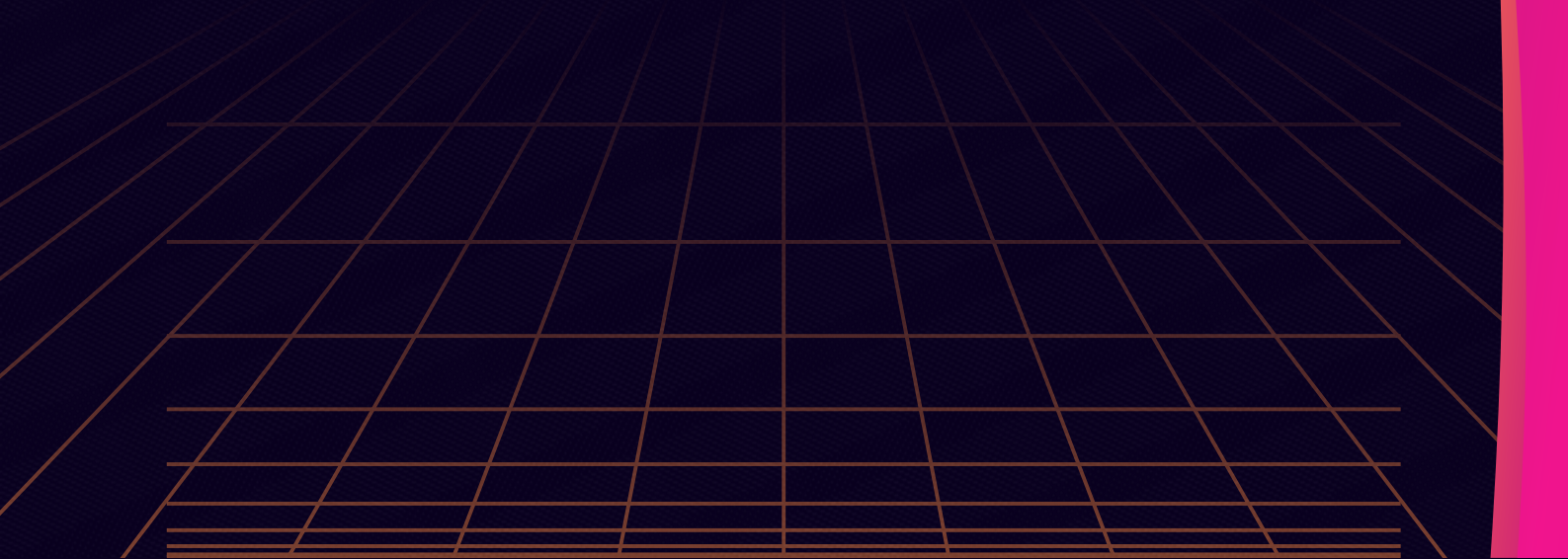
INTENT	BASH	PYTHON
First argument	`$1`	`sys.argv[1]`
All arguments	`"$@"`	`sys.argv[1:]`
Interpolation	`"$name"`	`f"{name}"`
File exists	`[[ -e $p ]]`	`Path(p).exists()`
Exit 0	`exit 0`	`return 0` from `main()`
Source a file	`source lib.sh`	`import module`
Command substitution	`$(cmd)`	`subprocess.run(...)`
Cleanup on exit	`trap cleanup EXIT`	`try/finally` or a context manager
Glob	`*.log`	`Path(".").glob("*.log")`
Pipe	`a | b`	A generator feeding a generator

• **READ THAT TABLE AS INTENT, NOT EQUIVALENCE**

An imported module is a cached object, not textual inclusion. A generator pipeline is lazy cooperation inside one process, not kernel pipes between two.

12 MATCH, PROPERLY

`match` is not a switch. It destructures. That is the whole reason it exists, and using it as a chain of equality tests is a waste of it.



```
match command.split():
    case ["get", key]:
        return store[key]
    case ["set", key, value]:
        store[key] = value
    case ["del", *keys] if keys:
        for key in keys:
            store.pop(key, None)
    case []:
        raise ValueError("empty command")
    case _:
        raise ValueError(f"unknown: {command!r}")
```

PATTERN	MATCHES
`case 200:`	The literal value
`case "a" | "b":`	Either alternative
`case [x, y]:`	A two-item sequence, binding both
`case [first, *rest]:`	At least one item; `rest` gets a list
`case {"type": "file", "size": n}:`	A mapping containing those keys; extra keys allowed
`case Point(x=0, y=y):`	A class instance, matching one attribute, binding the other
`case str() | bytes():`	By type, with no attribute constraints
`case x if x > 100:`	A guard – bind, then test
`case _:`	Anything. The wildcard, always last

• TRAP – BARE NAMES ALWAYS MATCH

`case ERROR:` does not compare against a constant named `ERROR`. It **binds** whatever came in to the name `ERROR` and matches everything. Use a dotted name – `case Status.ERROR:` – to compare against a constant. This eats an afternoon the first time.

13 COMPREHENSIONS AND WHEN TO STOP

```
squares  = [n * n for n in range(10)]
by_name  = {p.name: p.stat().st_size for p in paths if p.is_file()}
suffixes = {p.suffix for p in paths}
lazy     = (line.strip() for line in handle)     # generator, not a list
```

```
matrix  = [[1, 2], [3, 4]]
flat    = [x for row in matrix for x in row]     # loops read left to
right
labels  = ["big" if n > 10 else "small" for n in sizes]   # ternary goes
first
```

• WHERE TO STOP

A comprehension is a compact transformation, not a contest. Two `for` clauses plus a condition is the practical ceiling. Past that, or the moment a side effect appears, write the loop. Never use a comprehension purely for its side effect – that is a `for` statement wearing a disguise.

14 FUNCTIONS ARE CONTRACTS

```
def format_size(size: int) -> str:
    """Return a compact binary-unit representation of a byte count."""
    if size < 0:
        raise ValueError("size cannot be negative")

    units = ("B", "KiB", "MiB", "GiB", "TiB")
    value = float(size)
    for unit in units:
        if value < 1024 or unit == units[-1]:
            return f"{value:.1f} {unit}"
        value /= 1024

    raise AssertionError("unreachable")
```

CONTRACT TERM	HERE
Input	A byte count as `int`
Output	A human-readable `str`
Mutation	None
I/O	None
Failure	`ValueError` on a negative size — a decision, not an accident

14.1 The full parameter grammar

```
def f(pos_only, /, normal, *args, kw_only, **kwargs) -> None:
    ...
```

Before the /

> Positional-only. Callers cannot use the name, so you are free to rename it later.

Between / and *

> Ordinary: positional or keyword.

***args**

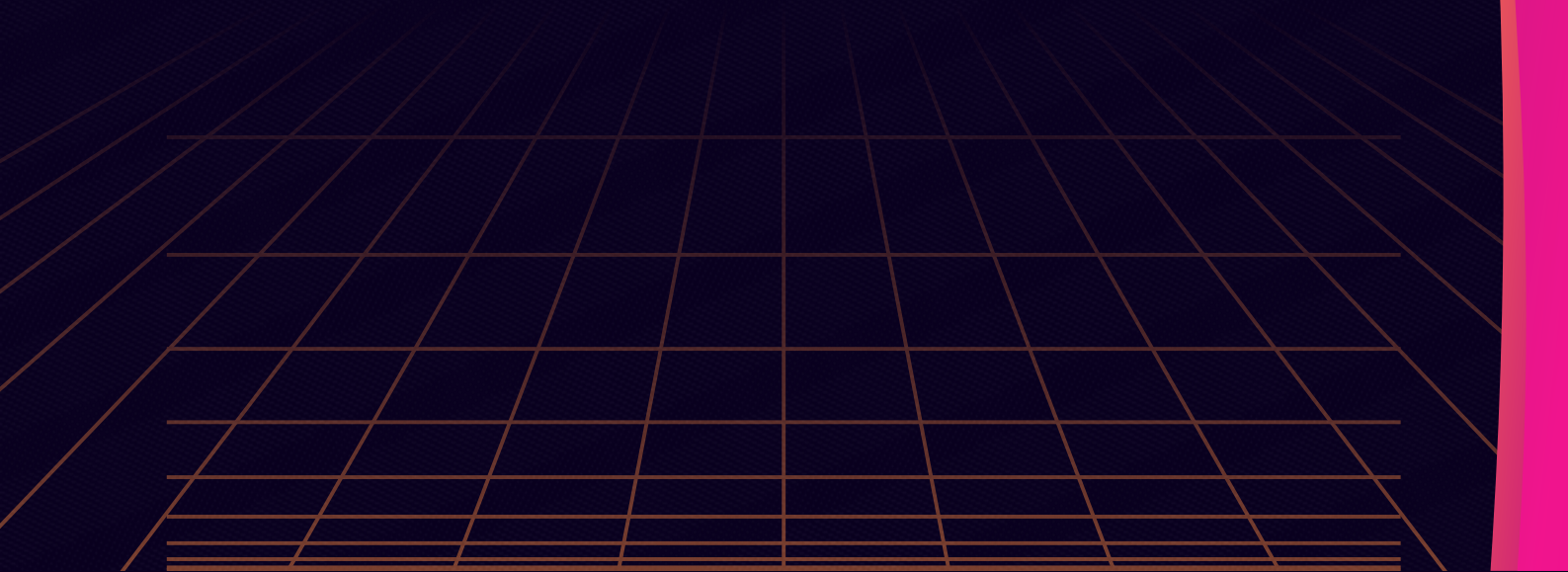
> Collects surplus positional arguments into a tuple.

After * or *args

> Keyword-only. This is how you forbid `describe(path, True)`.

****kwargs**

> Collects surplus keyword arguments into a dict.



• THE CLASSIC TRAP

```
def remember(name, names=[]):     # ONE list, created at def time
    names.append(name)
    return names
```

Defaults are born once, when the `def` executes. Anything mutable there is shared furniture. The fix is always the same:

```
def remember(name: str, names: list[str] | None = None) ->
list[str]:
    if names is None:
        names = []
    names.append(name)
    return names
```

• RETURN IS NOT PRINT

`return` hands a value to the caller. `print` writes to a stream and returns `None`. Keep computation separate from presentation – a function that prints cannot be tested, reused, or piped into anything.

15 SCOPE, CLOSURES, LEGB

Lookup climbs outward: **L**ocal, **E**nclosing, **G**lobal, **B**uilt-in. Assignment does not climb – it plants a local flag.

```
count = 0

def tick():
    count += 1        # UnboundLocalError: assignment made it local
```

```
def tick():
    global count      # explicit, and usually a smell
    count += 1
```

```
def make_counter():
    total = 0
    def tick() -> int:
        nonlocal total
        total += 1
        return total
    return tick

c = make_counter()
c(); c()              # 1, then 2 — the enclosing name survived the return
```

• TRAP – LATE BINDING IN LOOPS

```
fns = [lambda: i for i in range(3)]
[f() for f in fns]                 # [2, 2, 2]  — not [0, 1, 2]

fns = [lambda i=i: i for i in range(3)]
[f() for f in fns]                 # [0, 1, 2]  — default captures
now
```

A closure captures the *name*, not the value at creation time.

16 EXCEPTIONS AS TYPED FAILURE PATHS

```
try:
    stat_result = path.stat()
except FileNotFoundError:
    warn(f"vanished: {path}")
except PermissionError:
    warn(f"permission denied: {path}")
else:
    use(stat_result)          # runs only if no exception
finally:
    tidy()                    # runs no matter what
```

• NEVER

```
try:
    ...
except Exception:
    pass
```

That destroys evidence and converts corruption into apparent success. Catch the narrowest type you can actually handle, at the narrowest boundary that raises it.

16.1 The hierarchy you need

```
BaseException
├── SystemExit              raise SystemExit(2) — do not catch casually
├── KeyboardInterrupt       Ctrl-C — do not swallow
└── Exception               catch THIS if you must catch broadly
    ├── ArithmeticError → ZeroDivisionError
    ├── LookupError     → IndexError, KeyError
    ├── OSError         → FileNotFoundError, PermissionError,
    │                     IsADirectoryError, TimeoutError
    ├── ValueError      → right type, wrong value
    ├── TypeError       → wrong type entirely
    ├── AttributeError  → no such attribute
    ├── NameError       → no such name
    └── RuntimeError    → StopIteration, RecursionError
```

• RAISE YOUR OWN, AND CHAIN

```
class ConfigError(Exception):
    """Raised when the config file is unusable."""

try:
    data = json.loads(text)
except json.JSONDecodeError as error:
    raise ConfigError(f"{path}: invalid JSON") from error
```

`from error` preserves the original traceback under "The above exception was the direct cause." Use `from None` to deliberately hide a noisy internal cause.

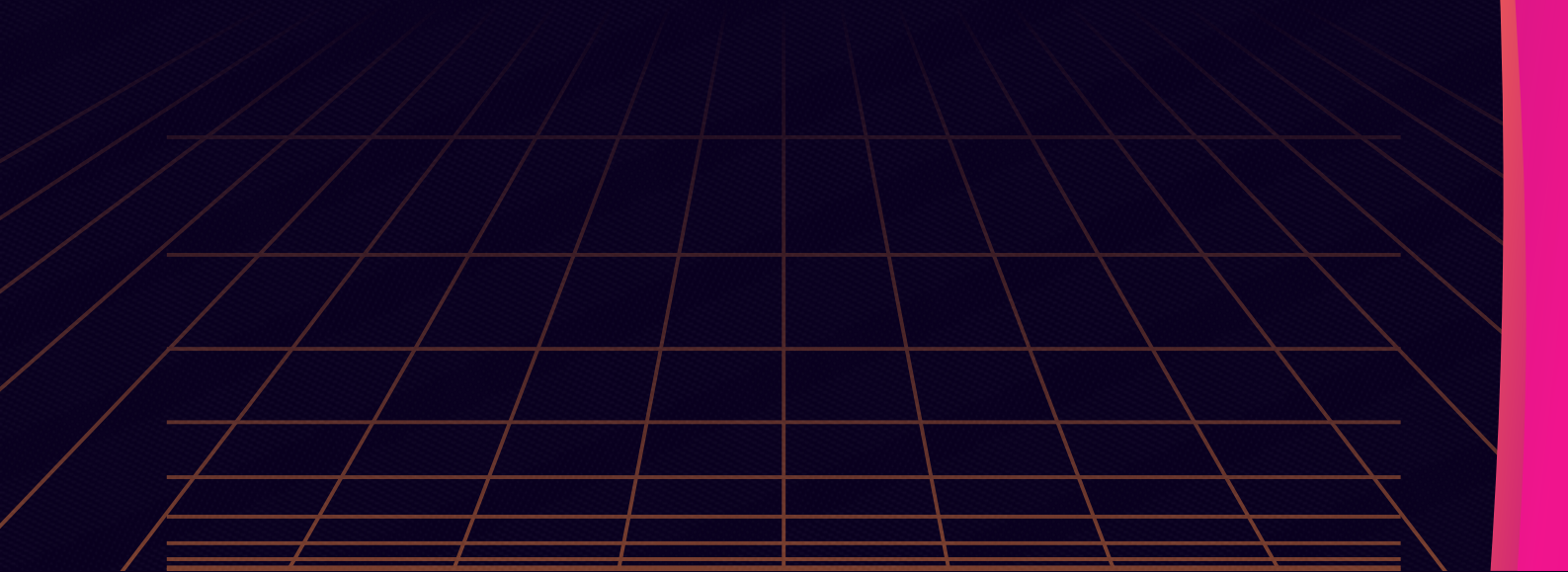
• CHALLENGE LEVEL 2

Write `parse_size("4KiB") -> int`. It accepts `B`, `KiB`, `MiB`, `GiB`, case-insensitively, with or without a space. It raises `ValueError` with a useful message on anything else. Then write the five calls that prove it, including two that must raise.

Hint: one of `str.removesuffix`, `str.partition`, or a single regex will do the parsing. The hard part is not parsing — it is deciding what `"4 kib"`, `"4"`, `""` and `"-4KiB"` each mean, before you write a line.

PROGRESS — PART II

- [ ] Explained rebinding vs. mutation out loud
- [ ] Reproduced the shallow-copy surprise deliberately
- [ ] Found a real place where `or` would eat a valid zero
- [ ] Wrote a `match` that destructures rather than compares
- [ ] Hit `UnboundLocalError` on purpose and explained it
- [ ] Reproduced the late-binding closure bug and fixed it two ways
- [ ] Raised a custom exception with `from error`
- [ ] Completed the Level 2 challenge



PART III

III

OBJECTS AND TYPES

Everything is an object, including classes, functions and modules. This part is about choosing the right shape for data — and about the machinery underneath, so that `@dataclass` stops being magic.

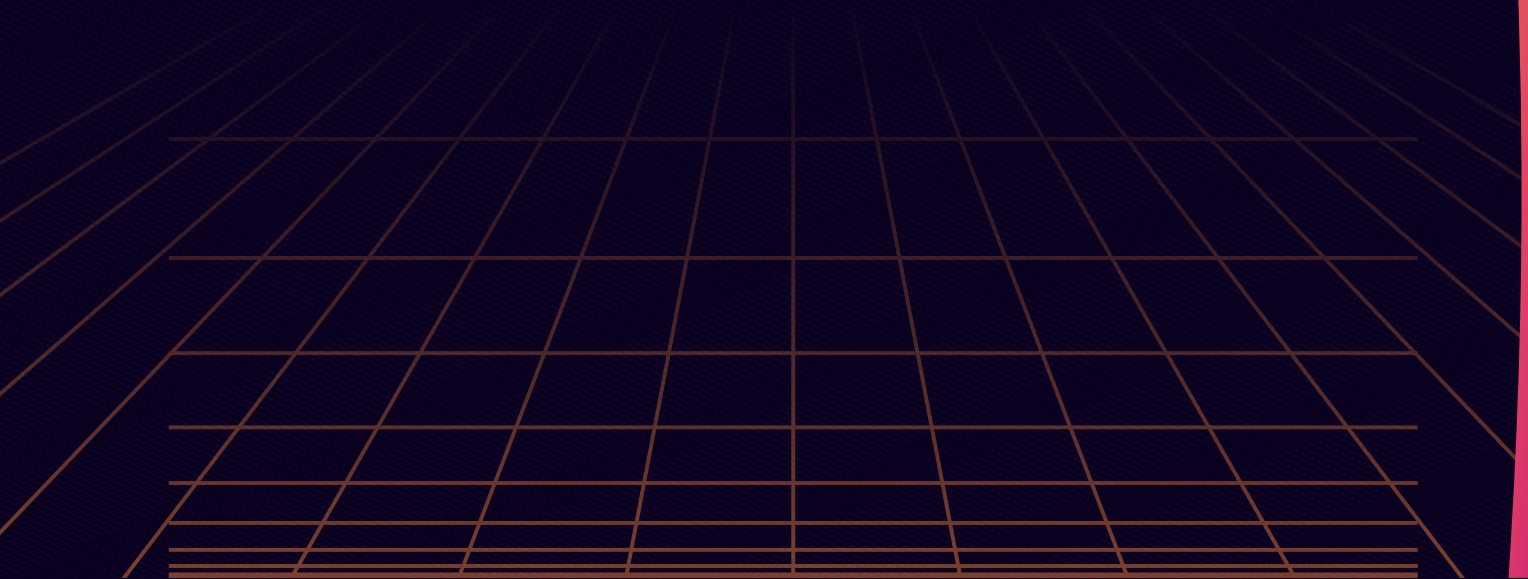
17 THE CORE TYPES

```
count   = 3                  # int      arbitrary precision
ratio   = 0.75               # float    IEEE 754 double
label   = "report"           # str      immutable text
payload = b"\x00\xff"        # bytes    immutable binary
flag    = True               # bool     an int subclass
nothing = None               # NoneType singleton

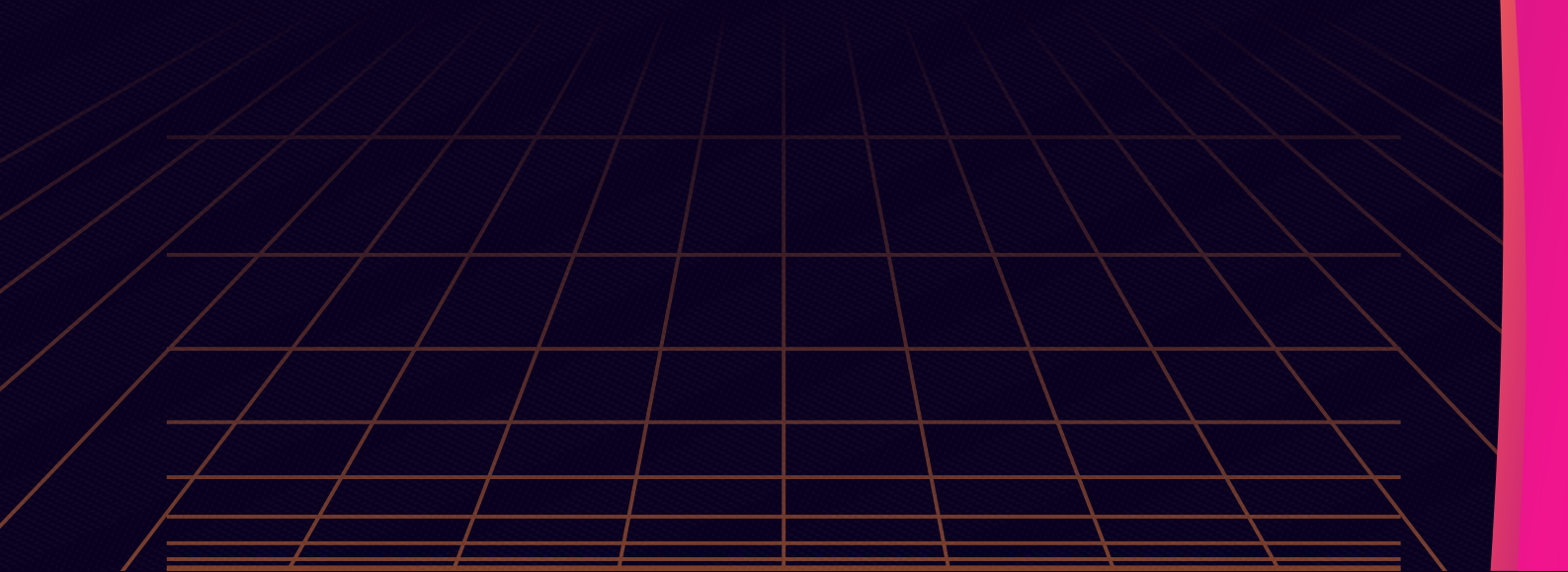
names  = ["a", "b"]          # list     ordered, mutable
point  = (10, 20)            # tuple    ordered, immutable, hashable
unique = {"py", "sh"}        # set      unordered, unique, mutable
record = {"path": "/tmp/x"}  # dict     insertion-ordered mapping
```

• THE ONLY QUESTION THAT MATTERS

Not “which bracket do I type?” but “what behaviour does this data need?”



NEED	REACH FOR	BECAUSE
Ordered, changes over time	`list`	Indexable, mutable, cheap append
Fixed positional record	`tuple`	Immutable, hashable, unpacks cleanly
"Have I seen this?"	`set`	O(1) membership, dedupes for free
Look up by meaningful key	`dict`	Named access, JSON-shaped
Named record with types	`dataclass`	Real attributes, typos fail loudly
Fixed set of choices	`enum.Enum`	Comparable constants with names
Counting occurrences	`collections.Counter`	Counting and `.most_common()` built in
Grouping into lists	`defaultdict(list)`	No "if key not in" dance
Queue from both ends	`collections.deque`	O(1) at both ends; `maxlen` ring buffer



17.1 The string methods you will actually use

```
s.strip()  s.lstrip()  s.rstrip()       # whitespace or given chars
s.split()  s.split(",", maxsplit=1)     # no-arg split handles runs of
whitespace
s.rsplit(",", 1)                        # from the right
s.partition("=")                        # ('key', '=', 'value') — always 3
items
s.join(items)                           # "\n".join(lines)
s.startswith(("#", "//"))               # a tuple tests several at once
s.removeprefix("v")  s.removesuffix(".log")
s.casefold()                            # aggressive lower(), correct for
comparison
s.encode("utf-8")   b.decode("utf-8")   # the str/bytes boundary
s.zfill(3)  s.ljust(20)  s.rjust(12)
s.replace("\t", "    ", 1)              # count is optional
s.splitlines()                          # handles \n, \r\n, and the odd
ones
```

• PARTITION BEATS SPLIT FOR KEY=VALUE

`"a=b=c".split("=")` gives three items and breaks your unpacking. `"a=b=c".partition("=")` always gives exactly three: before, separator, after. It also tells you whether the separator was there at all – the middle is empty if not.

17.2 Dict methods that remove branches

```
d.get(key)                  # None instead of KeyError
d.get(key, fallback)
d.setdefault(key, []).append(x)
d.pop(key, None)            # remove if present, never raise
d | other                   # merged copy (3.9+)
d |= other                  # update in place
d.items()  d.keys()  d.values()
{**a, **b}                  # older merge idiom, still common
```

18 DATACLASSES

The default answer for “a record with named fields.”

```python
from dataclasses import dataclass, field
from datetime import UTC, datetime
from pathlib import Path


@dataclass(frozen=True, slots=True)
class FileRecord:
    path: Path
    size: int
    modified: datetime
    tags: list[str] = field(default_factory=list)

    @classmethod
    def from_path(cls, path: Path) -> "FileRecord":
        st = path.stat()
        return cls(
            path=path,
            size=st.st_size,
            modified=datetime.fromtimestamp(st.st_mtime, tz=UTC),
        )

    @property
    def is_large(self) -> bool:
        return self.size > 1_000_000

    def to_jsonable(self) -> dict[str, object]:
        return {
            "path": str(self.path),
            "size": self.size,
            "modified": self.modified.isoformat(timespec="seconds"),
        }
```

@dataclass

> A decorator. It receives the class and returns it with `__init__`, `__repr__` and `__eq__` generated from the annotations.

frozen=True

> Blocks field rebinding after construction, and makes instances hashable.

slots=True

> Fixed instance layout: less memory, faster attribute access, and a typo'd attribute raises instead of silently creating a new one.

field(default_factory=list)

The correct way to default to a mutable. A bare `= []` is rejected by dataclasses outright — the one place Python protects you from that trap.

@classmethod from_path

An alternate constructor, because the raw input is a path but building the record needs filesystem work.

@property

A method that reads like an attribute. Use it for cheap derived values, never for anything slow or surprising.

19 CLASSES, AND WHEN NOT TO

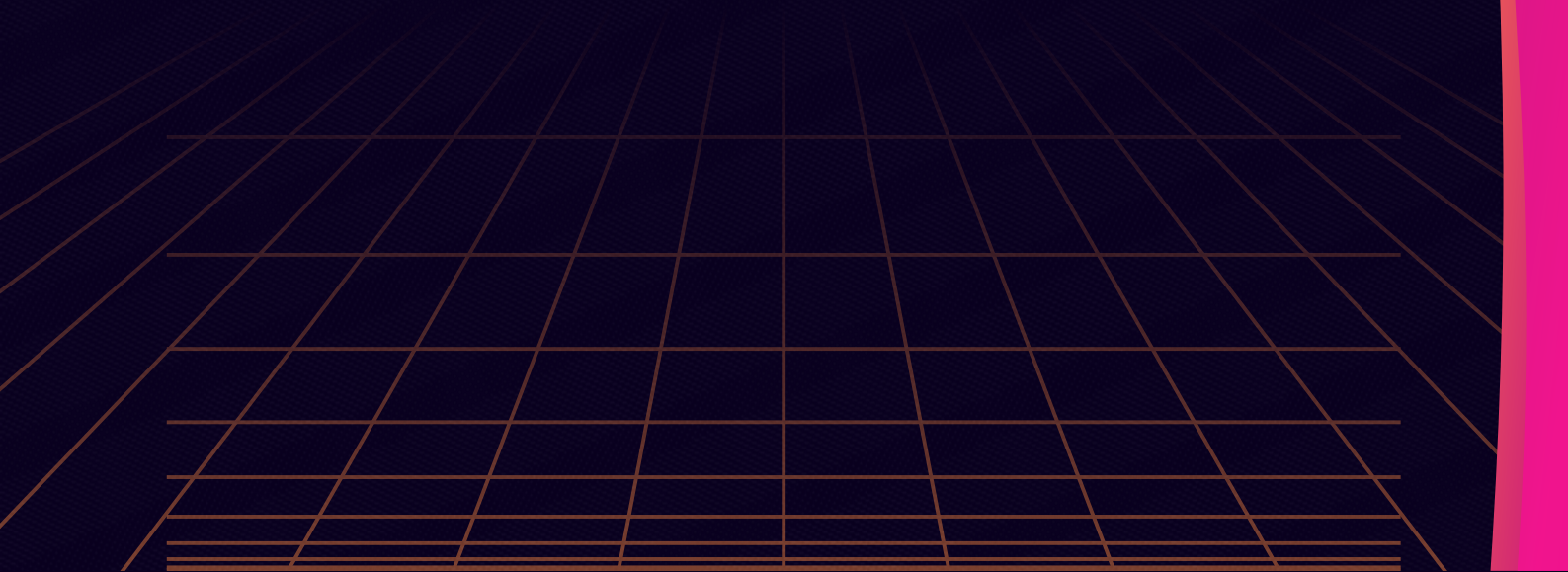
• KEY IDEA

Use a class when data and behaviour form a durable concept, when instances hold independent state, or when callers benefit from swapping implementations. A class is not automatically "more advanced" than a function.

```
class SizeFormatter:            # ceremony
    def format(self, size): ...

def format_size(size): ...      # the entire concept
```

But this earns it, because configuration and accumulated state genuinely span calls:



```
class Scanner:
    def __init__(self, include_hidden: bool = False) -> None:
        self.include_hidden = include_hidden
        self.visited = 0
        self.failed = 0

    def scan(self, root: Path) -> Iterator[FileRecord]:
        for path in walk_files(root):
            if not self.include_hidden and path.name.startswith("."):
                continue
            self.visited += 1
            try:
                yield FileRecord.from_path(path)
            except OSError:
                self.failed += 1
```

THE TRADEOFF YOU JUST ACCEPTED

Hidden mutation. Callers must know that consuming `scan()` is what changes `visited` – and that reading the counter before exhausting the generator gives a meaningless number.

METHOD KIND	RECEIVES	USE WHEN
Instance	`self`	It needs this object's state
`@classmethod`	`cls`	Alternate constructors, mostly
`@staticmethod`	Neither	Rarely – a module function is usually clearer
`@property`	`self`	Cheap derived value that reads as data

20 __SLOTS__, DESCRIPTORS, METACLASSES

The machinery under the syntax. You will not write these often; you need to recognise them, because libraries you depend on are built from them.

20.1 __slots__

```
class Point:
    __slots__ = ("x", "y")
    def __init__(self, x, y):
        self.x, self.y = x, y

p = Point(1, 2)
p.z = 3        # AttributeError — no __dict__ to put it in
```

Normally every instance carries a `__dict__`. `__slots__` replaces it with fixed storage: smaller, faster, and typo-proof. The cost is no dynamic attributes and some inheritance care.

20.2 Descriptors

An object that defines `__get__`, `__set__` or `__delete__`, used as a class attribute. This is what `property`, `classmethod` and `staticmethod` *are*.

```
class Positive:
    def __set_name__(self, owner, name):
        self.name = "_" + name

    def __get__(self, obj, objtype=None):
        if obj is None:
            return self
        return getattr(obj, self.name)

    def __set__(self, obj, value):
        if value < 0:
            raise ValueError(f"{self.name[1:]} must be >= 0")
        setattr(obj, self.name, value)


class Item:
    size = Positive()          # validation lives in the descriptor, once

item = Item()
item.size = 10                 # fine
item.size = -1                 # ValueError
```

• WHY THEY EXIST

A descriptor moves attribute behaviour out of every class that needs it and into one reusable object. Write one when the same validation or lazy-loading appears on many attributes. Otherwise use `@property`.

20.3 Metaclasses

A class is an object, so something must have built it. That something is its metaclass – `type`, unless told otherwise.

```
Thing = type("Thing", (), {"kind": "demo"})   # a class, built at runtime

class Registry(type):
    registry = {}
    def __new__(mcls, name, bases, namespace):
        cls = super().__new__(mcls, name, bases, namespace)
        if bases:                          # skip the base itself
            Registry.registry[name.lower()] = cls
        return cls

class Plugin(metaclass=Registry):
    pass

class Csv(Plugin):  ...
class Json(Plugin): ...

Registry.registry          # {'csv': <class Csv>, 'json': <class Json>}
```

• THE HONEST ADVICE

Almost every metaclass you are tempted to write should be `__init_subclass__` instead – same auto-registration, a tenth of the complexity:

```
class Plugin:
    registry = {}
    def __init_subclass__(cls, **kwargs):
        super().__init_subclass__(**kwargs)
        Plugin.registry[cls.__name__.lower()] = cls
```

Know metaclasses so you can read Django, SQLAlchemy and pydantic. Reach for them roughly never.

21 TYPE HINTS IN REAL DEPTH

Annotations do not enforce anything at runtime. They are for readers, editors, and a type checker that reads your whole program at once.

```
def tally(paths: list[Path], limit: int | None = None) -> dict[str, int]:
    ...
```

```
from collections.abc import Callable, Iterable, Iterator, Mapping,
Sequence

def emit(rows: Iterable[str]) -> None: ...        # accept the widest
def build() -> list[str]: ...                     # return the most
specific
```

• THE RULE THAT MAKES ANNOTATIONS USEFUL

Be liberal in what you accept, specific in what you return. `Iterable[Path]` as a parameter accepts a list, tuple, set or generator. `list[Path]` as a parameter refuses all but one, for no benefit.

21.1 Generics and TypeVar

```
# 3.12+ syntax
def first[T](items: Sequence[T], default: T | None = None) -> T | None:
    return items[0] if items else default

class Box[T]:
    def __init__(self, item: T) -> None:
        self.item = item
    def get(self) -> T:
        return self.item

# pre-3.12 equivalent
from typing import TypeVar
T = TypeVar("T")
def first(items: Sequence[T], default: T | None = None) -> T | None: ...
```

The value of `T` is the link: it says the thing coming out is the same type as the thing that went in, so `first(["a"])` is known to be `str | None` and not `Any`.

21.2 The rest of the vocabulary

WRITTEN	MEANS
`int | None`	Either. `Optional[int]` is the old spelling
`list[str]`	A list of strings
`dict[str, int]`	Keys and values
`tuple[int, str]`	Exactly two items, those types
`tuple[int, ...]`	Any number, all `int`
`Callable[[int, str], bool]`	Takes those, returns that
`Literal["r", "w"]`	Only these exact values
`Final`	Do not rebind this
`Any`	Checking off. Use sparingly and deliberately
`TYPE_CHECKING`	Import only for the checker, avoiding a runtime cycle
`type Rows = list[dict[str, str]]`	A named alias (3.12+)
`cast(X, value)`	"Trust me." Zero runtime effect

```
python3 -m pip install mypy
mypy --strict yourmodule.py
```

22 PROTOCOLS OVER ANCESTRY

Python depends on behaviour, not inheritance. Iteration asks for `__iter__`; context managers provide `__enter__` and `__exit__`; path-likes provide `__fspath__`. These conventions are protocols, and you can state one explicitly:

```
from typing import Protocol

class Renderer(Protocol):
    def __call__(self, record: FileRecord) -> str: ...

def emit(records: Iterable[FileRecord], render: Renderer) -> None:
    for record in records:
        print(render(record))
```

Any callable of the right shape satisfies it. Nothing inherits from `Renderer`, and nothing needs to.

23 DECORATORS AND CONTEXT MANAGERS

A decorator is a call, performed after the decorated object is created:

```
@decorator
def function(): ...

# is roughly
def function(): ...
function = decorator(function)
```

```
import functools, time

def timed(fn):
    @functools.wraps(fn)                    # keeps __name__, __doc__,
signature
    def wrapper(*args, **kwargs):
        start = time.monotonic()
        try:
            return fn(*args, **kwargs)
        finally:
            print(f"{fn.__name__}: {time.monotonic() - start:.3f}s")
    return wrapper

@timed
def scan(root): ...
```

• ALWAYS @FUNCTOOLS.WRAPS

Without it the decorated function reports itself as `wrapper`, breaking tracebacks, `help()`, and anything that introspects. One line, and you never debug that confusion.

23.1 Stdlib decorators worth knowing

```
@functools.cache                # memoise, unbounded
@functools.lru_cache(maxsize=256)
@functools.cached_property      # computed once per instance, then an
attribute
@dataclasses.dataclass
@contextlib.contextmanager
@staticmethod  @classmethod  @property
```

23.2 Context managers

```
from contextlib import contextmanager

@contextmanager
def timed(label: str):
    start = time.monotonic()
    try:
        yield                    # the with-body runs here
    finally:
        print(f"{label}: {time.monotonic() - start:.3f}s",
file=sys.stderr)

with timed("scan"):
    run_scan()
```

• MEMORY HOOK

Before the `yield` is setup. After it is teardown. Put the teardown in `finally` or an exception in the body will skip it.

```
from contextlib import suppress
with suppress(FileNotFoundError):
    path.unlink()                # narrow, explicit, and readable
```

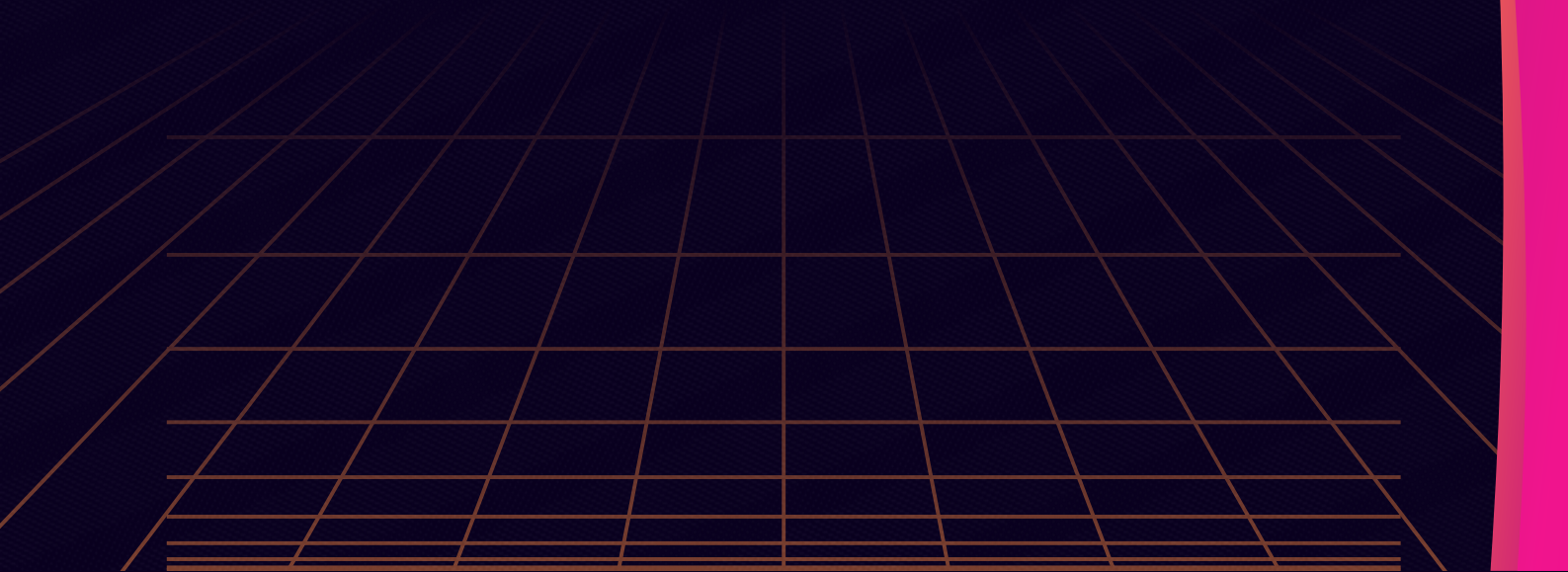
• CHALLENGE LEVEL 3

Build `@retry(times=3, delay=0.5, on=(TimeoutError,))`. It must preserve the wrapped function's identity, re-raise the last exception when attempts run out, never swallow `KeyboardInterrupt`, and work on a method as well as a plain function.

Hint: a decorator that takes arguments is a function returning a decorator – three levels of `def`. Test it with a function that fails twice and then succeeds, and assert on the call count.

PROGRESS – PART III

- [ ] Chose a container by behaviour, not habit, three times
- [ ] Replaced an if/else block with `dict.get` or `setdefault`
- [ ] Wrote a frozen, slotted dataclass with an alternate constructor
- [ ] Triggered the `__slots__` `AttributeError` on purpose
- [ ] Wrote a descriptor, then rewrote it as a `@property`
- [ ] Replaced a metaclass idea with `__init_subclass__`
- [ ] Ran `mypy --strict` on something of mine
- [ ] Completed the Level 3 challenge



PART IV

IV LAZINESS

The difference between a program that handles a directory and a program that handles a filesystem. Three sections, and they change how you write everything afterwards.

24 ITERABLES, ITERATORS, GENERATORS

THREE WORDS, THREE MEANINGS

An **iterable** can produce an iterator (`__iter__`). An **iterator** produces one item at a time and is exhausted once (`__next__`). A generator is the easiest way to write an iterator.

```
names = ["a", "b", "c"]
it = iter(names)
next(it)          # 'a'
next(it)          # 'b'
next(it, None)    # 'c'   — the default prevents StopIteration
```

```
def walk_files(root: Path) -> Iterator[Path]:
    for path in root.rglob("*"):
        if path.is_file():
            yield path
```

Calling `walk_files(root)` scans nothing. It returns a generator object. Work happens only as the caller pulls:

```
all_paths  = list(walk_files(root))   # eager: the whole tree in memory
first_path = next(walk_files(root))   # exactly enough work for one result
```

• TRAP – ITERATORS ARE ONE-SHOT

```
records = collect(paths)
count = sum(1 for _ in records)   # consumes it
for r in records:                 # silently iterates nothing
    print(r)
```

A list can be walked twice; a generator cannot. If you need both, materialise deliberately with `list()` and accept the memory.

24.1 Writing your own iterator class

```
class Countdown:
    def __init__(self, start: int) -> None:
        self.current = start

    def __iter__(self):
        return self

    def __next__(self) -> int:
        if self.current <= 0:
            raise StopIteration
        self.current -= 1
        return self.current + 1
```

That is what the `for` statement calls. A generator function writes the same thing in four lines – which is why you almost always use one.

25 GENERATORS BEYOND THE BASICS

25.1 yield from

```
def walk(root: Path) -> Iterator[Path]:
    for child in root.iterdir():
        if child.is_dir():
            yield from walk(child)     # delegate to the sub-generator
        else:
            yield child
```

Without `yield from` that inner case is a loop that re-yields every item. With it, delegation is one line and recursion reads naturally.

25.2 Generator pipelines

```
def read_lines(path: Path) -> Iterator[str]:
    with path.open(encoding="utf-8", errors="replace") as fh:
        yield from fh

def strip_blank(lines: Iterable[str]) -> Iterator[str]:
    for line in lines:
        line = line.rstrip("\n")
        if line.strip():
            yield line

def only_errors(lines: Iterable[str]) -> Iterator[str]:
    for line in lines:
        if "ERROR" in line:
            yield line

for line in only_errors(strip_blank(read_lines(path))):
    print(line)
```

• WHY THIS MATTERS

It looks like a shell pipeline, but every stage is an iterator in one process — nothing is serialised, nothing is buffered to disk. Backpressure is automatic: a stage does work only when the stage after it asks. This handles a 40 GB log on a laptop.

• TRAP — THE FILE CLOSES EARLY

In `read_lines` the `with` block stays open only while the generator is alive. Abandon it half-way and the handle closes at garbage-collection time, not deterministically. Consume generators that own resources inside the scope where you created them.

25.3 Sending values in

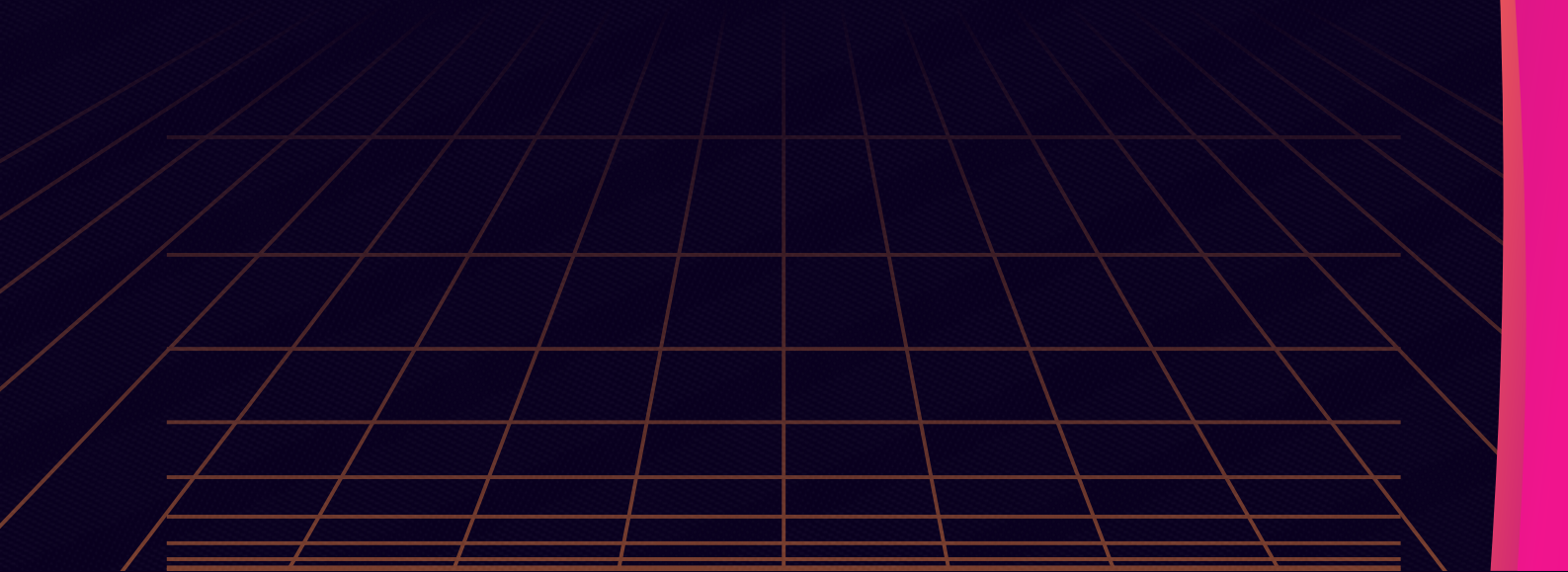
```
def accumulator():
    total = 0
    while True:
        value = yield total      # yield produces AND receives
        total += value

acc = accumulator()
next(acc)        # prime it: 0
acc.send(10)     # 10
acc.send(5)      # 15
```

Rarely the right tool — a class with a method is usually clearer — but this is the mechanism that coroutines were built on, so recognising it pays off later.

26 ITERTOOLS AS A VOCABULARY

Every one of these is lazy. Learning them replaces a lot of hand-written loops.



CALL	YIELDS
`chain(a, b)`	a's items, then b's — no concatenation
`chain.from_iterable(lists)`	Flattens one level
`islice(it, 10)`	First 10; `islice(it, 5, 15)` for a window
`takewhile(pred, it)`	Items until the first failure, then stops
`dropwhile(pred, it)`	Skips the leading run, yields the rest
`groupby(sorted_it, key)`	`(key, group)` — **requires pre-sorted input**
`batched(it, 100)`	Tuples of 100 (3.12+) — ideal for bulk inserts
`pairwise(it)`	`(a,b), (b,c), (c,d)` — deltas between rows
`count(1)` `cycle(it)` `repeat(x, 3)`	Infinite; always pair with `islice` or a `break`
`zip_longest(a, b, fillvalue=0)`	Pads instead of truncating
`product / permutations / combinations`	Combinatorics, lazily
`tee(it, 2)`	Two independent iterators — buffers, so beware

• TRAP – GROUPBY

`itertools.groupby` groups *consecutive* equal keys. Unsorted input gives you the same key several times and looks broken. Sort by the same key first, or use `defaultdict(list)` instead. Also: each group is itself a lazy iterator, invalid once you advance to the next one.

• WORKED EXAMPLE – TEN BIGGEST FILES, WITHOUT A LIST OF EVERYTHING

```
import heapq

def largest(paths: Iterable[Path], n: int = 10) -> list[tuple[int,
Path]]:
    sized = ((p.stat().st_size, p) for p in paths)
    return heapq.nlargest(n, sized, key=lambda pair: pair[0])
```

`sorted(...)[:10]` would build the entire list first. `heapq.nlargest` keeps only ten in memory no matter how many files stream past.

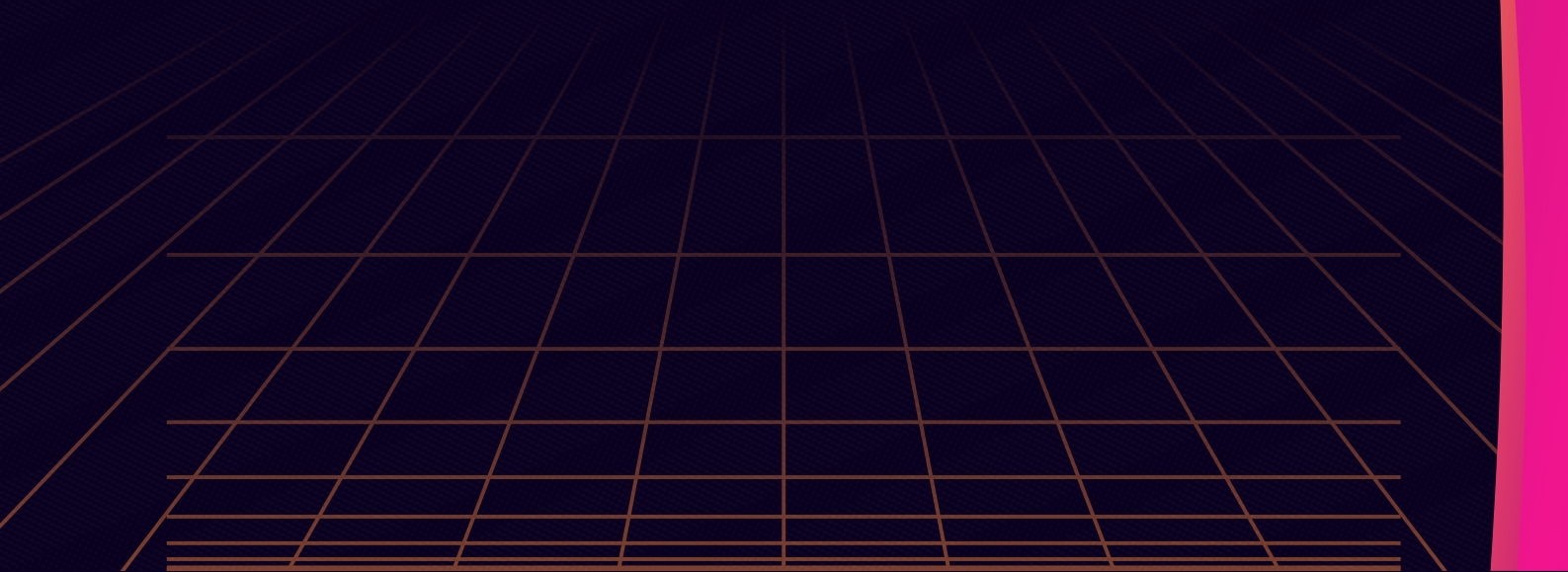
• CHALLENGE LEVEL 3

Write `tail(path, n)` that returns the last `n` lines of a file without reading the whole file into memory and without seeking. Then write `follow(path)`, a generator that yields new lines as they are appended, like `tail -f`.

Hint: for the first, one `collections.deque` with `maxlen` does it in three lines. For the second, you need a loop, a sleep, and a decision about what to do when the file is truncated or rotated underneath you.

PROGRESS – PART IV

- [ ] Reproduced the exhausted-iterator bug on purpose
- [ ] Wrote a three-stage generator pipeline
- [ ] Used `yield from` for recursive traversal
- [ ] Replaced a hand-written loop with an `itertools` call
- [ ] Hit the `groupby` sorting trap and understood it
- [ ] Completed `tail` and `follow`



PART V

V

REGULAR EXPRESSIONS

A second language living inside the first. Learned properly once, it pays out in every log file, config parser and scraper you will ever write.

27 THE SYNTAX, COMPLETELY

27.1 Single characters

PATTERN	MATCHES
`.`	Any character except newline (any at all with `re.S`)
`\d` `\D`	A digit / not a digit
`\w` `\W`	Word character `[a-zA-Z0-9_]` / not
`\s` `\S`	Whitespace / not
`[abc]`	Any one of a, b, c
`[^abc]`	Any one character that is not a, b or c
`[a-z0-9_]`	Ranges inside a class
`\.` `\\` `\[`	Escaped literals

27.2 Repetition

PATTERN	MEANS
`*`	Zero or more, greedy
`+`	One or more, greedy
`?`	Zero or one
`{3}` `{2,5}` `{2,}`	Exactly / between / at least
`*?` `+?` `??` `{2,5}?`	The same, but lazy — shortest match

GREEDY VERSUS LAZY — THE CLASSIC

```
re.search(r"<(.+)>",  "<a><b>").group(1)    # 'a><b'   greedy
re.search(r"<(.+?)>", "<a><b>").group(1)    # 'a'      lazy
```

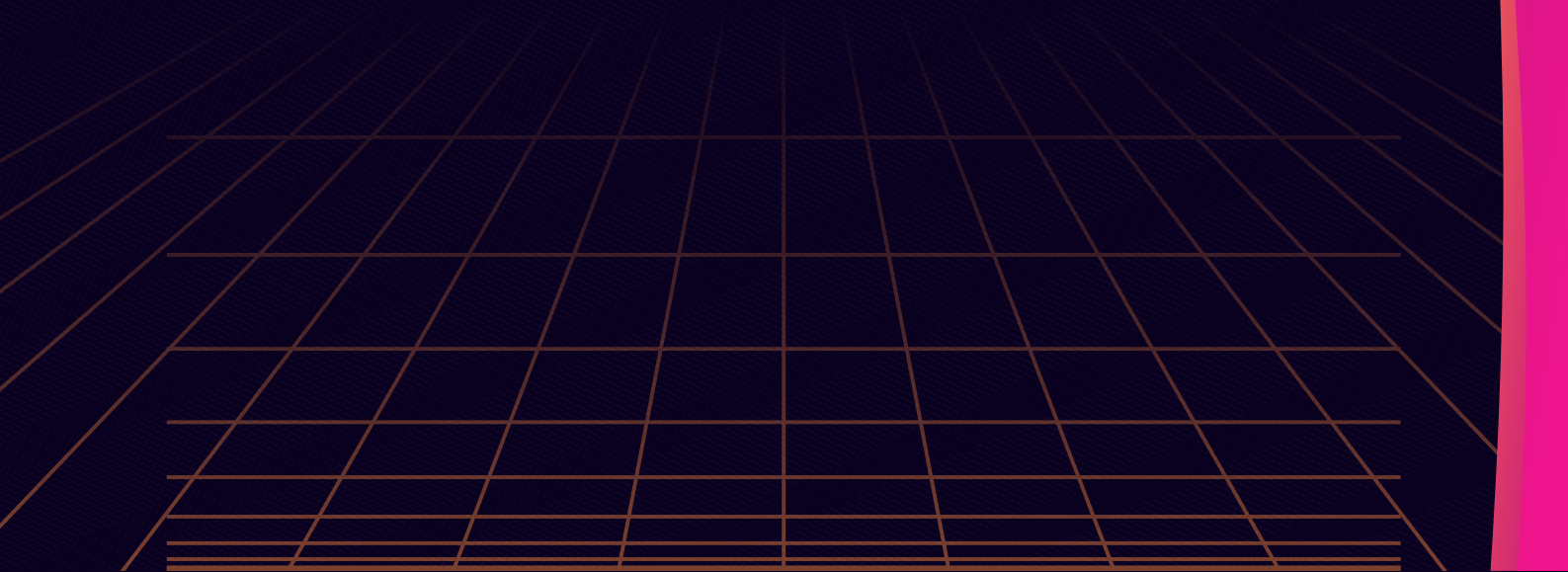
`.+` takes everything it can and gives back only as much as it must. Nine times in ten with delimiters, you want `.+?` — or better, `[^>]+`, which cannot overshoot at all.

27.3 Anchors, groups, alternation

PATTERN	MEANS
`^` `$`	Start / end of string — of each line with `re.M`
`\b` `\B`	Word boundary / not a boundary
`\A` `\Z`	Absolute start / end, ignoring `re.M`
`a|b`	Either; lowest precedence, so parenthesise
`(...)`	Capturing group, numbered from 1
`(?:...)`	Group without capturing
`(?P<name>...)`	Named capture — use these
`(?=...)` `(?!...)`	Lookahead: followed by / not followed by
`(?<=...)` `(?<!...)`	Lookbehind, fixed width only
`\1` `(?P=name)`	Backreference to an earlier group

27.4 Flags

```
re.I   IGNORECASE
re.M   MULTILINE    ^ and $ match at every line
re.S   DOTALL       . also matches newline
re.X   VERBOSE      whitespace and # comments allowed in the pattern
re.A   ASCII        \w \d \s stay ASCII-only
```



28 THE RE API

```
import re

m = re.search(pattern, text)       # first match anywhere, or None
m = re.match(pattern, text)        # anchored at the START only
m = re.fullmatch(pattern, text)    # must consume the whole string

re.findall(pattern, text)          # list of strings (or tuples if groups)
re.finditer(pattern, text)         # lazy iterator of match objects
re.sub(pattern, repl, text)        # replace; repl may be a function
re.subn(pattern, repl, text)       # (result, count)
re.split(r"[,;]\s*", text)         # split on a pattern
re.escape(user_input)              # treat user text as literal

rx = re.compile(pattern)           # compile once, reuse in a loop
```

```
if m := re.search(r"(?P<level>\w+):(?P<msg>.*)", line):
    print(m["level"], m["msg"])
    print(m.group(0))       # the whole match
    print(m.span())         # (start, end)
    print(m.groupdict())    # {'level': ..., 'msg': ...}
```

• TRAP – MATCH IS ANCHORED

`re.match` only ever matches at position zero. If you meant "find this anywhere," you want `re.search`. This is the single most common regex mistake in Python, and it fails silently by returning `None`.

29 WRITING REGEXES YOU CAN STILL READ

```
LOG = re.compile(r"""
    ^(?P<stamp>\d{4}-\d{2}-\d{2}[ T]\d{2}:\d{2}:\d{2})   # ISO-ish
timestamp
    \s+
    (?P<level>DEBUG|INFO|WARNING|ERROR|CRITICAL)
    \s+
    (?P<logger>[\w.]+)
    :\s*
    (?P<message>.*)$
""", re.VERBOSE)

for line in read_lines(path):
    if m := LOG.match(line):
        record = m.groupdict()
```

• THREE RULES THAT KEEP REGEX MAINTAINABLE

1. **Always** `r"..."`. Without it `\b` is a backspace, not a word boundary.
2. **Name your groups and use** `re.VERBOSE` the moment a pattern exceeds one line. `m["level"]` survives edits that renumber `m.group(3)`.
3. **Compile module-level constants** when the pattern runs in a loop.

• WHEN NOT TO USE REGEX AT ALL

CSV → `csv`. JSON → `json`. HTML → a parser, never a regex. URLs → `urllib.parse`. Config → `tomllib` or `configparser`. Fixed delimiters → `str.partition`. Regex is for irregular text you do not control – mostly logs.

• CATASTROPHIC BACKTRACKING

Nested quantifiers over overlapping character sets – `(\w+\s*)+$` – can take exponential time on a non-matching line. On attacker-supplied input that is a denial of service. Prefer specific classes (`[^,]+`) over `.+`, and never nest a quantifier inside a quantified group.

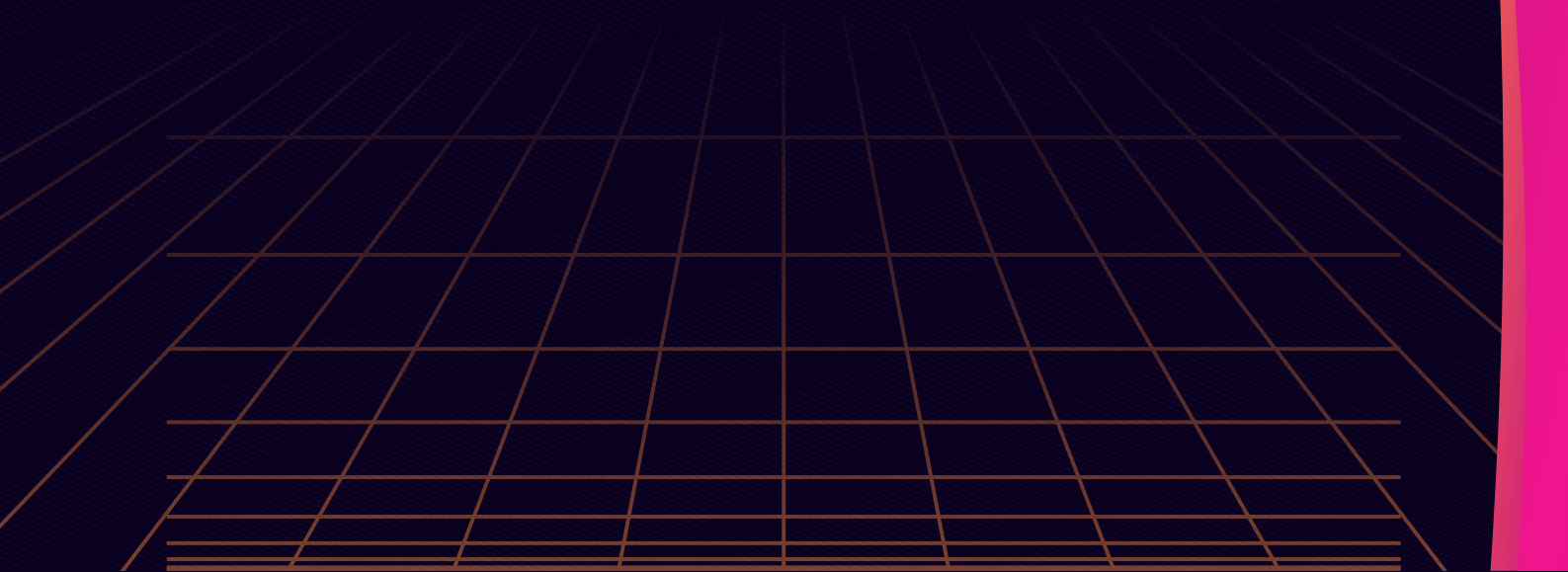
• CHALLENGE LEVEL 4

Parse a real nginx access log into records: IP, timestamp as an aware `datetime`, method, path, status, bytes, referrer, user-agent. Handle quoted fields containing escaped quotes, and `-` meaning "absent" in three different columns. Report the top ten paths by total bytes.

Hint: one `re.VERBOSE` pattern with named groups gets you the fields. Everything hard is afterwards: the timestamp format needs `%d/%b/%Y:%H:%M:%S %z`, and `-` must become `None` before you try to sum it. Stream the file; do not read it.

PROGRESS — PART V

- [ ] Can state the difference between `match` and `search` instantly
- [ ] Fixed a greedy quantifier that was eating too much
- [ ] Wrote a `re.VERBOSE` pattern with named groups
- [ ] Used `re.escape` on something a user typed
- [ ] Caught myself about to parse structured data with a regex
- [ ] Completed the nginx challenge



PART VI

VI

SKELETONS, SMALL TO LARGE

Six templates. Each is complete and runnable, each earns its extra structure, and each is a thing you should be able to type from memory. Growth is a decision, not a drift.

30 THE ONE-FILE SCRIPT

Level zero. Top to bottom, no functions, nothing clever. Correct for something you will run twice and delete.

```
#!/usr/bin/env python3
"""Count files by extension under a directory."""

import sys
from collections import Counter
from pathlib import Path

root = Path(sys.argv[1]) if len(sys.argv) > 1 else Path.cwd()

counts = Counter(
    p.suffix.lower() or "<none>"
    for p in root.rglob("*")
    if p.is_file()
)

for suffix, count in counts.most_common():
    print(f"{count:>7}  {suffix}")
```

• WHAT IT ALREADY GETS RIGHT

A shebang, a docstring, an argument with a sane default, and output that is aligned and pipeable. The flow is visible in one screen. Do not add a class to this.

31 THE FUNCTION-SHAPED SCRIPT

Promote when any of: it exceeds a screen, you want to test a piece, or you catch yourself copying a block.

```
#!/usr/bin/env python3
"""Count files by extension under a directory."""

import sys
from collections import Counter
from collections.abc import Iterable, Iterator, Sequence
from pathlib import Path


def walk_files(root: Path) -> Iterator[Path]:
    for path in root.rglob("*"):
        if path.is_file():
            yield path


def count_suffixes(paths: Iterable[Path]) -> Counter[str]:
    return Counter(p.suffix.lower() or "<none>" for p in paths)


def render(counts: Counter[str]) -> Iterator[str]:
    for suffix, count in counts.most_common():
        yield f"{count:>7}  {suffix}"


def main(argv: Sequence[str] | None = None) -> int:
    if argv is None:
        argv = sys.argv[1:]

    root = Path(argv[0]) if argv else Path.cwd()
    if not root.is_dir():
        print(f"not a directory: {root}", file=sys.stderr)
        return 2

    for line in render(count_suffixes(walk_files(root))):
        print(line)
    return 0
```

```
if __name__ == "__main__":
    raise SystemExit(main())
```

• THE SEAMS, AND WHY THEY ARE THERE

- `walk_files` – discovery, lazy, knows nothing about counting.
- `count_suffixes` – pure. No I/O, so it is trivially testable.
- `render` – presentation, yields strings rather than printing them.
- `main` – the only place that touches `argv`, `stderr`, and the exit status.

Under `if __name__ == "__main__"`, importing this module runs nothing. That one line is what makes the file testable.

32 THE ARGPARSE CLI

Promote when there is more than one input, or any flag at all. Never hand-parse `sys.argv` past the first argument.

```
#!/usr/bin/env python3
"""Report file metadata under one or more directories."""

import argparse
import sys
from collections.abc import Sequence
from pathlib import Path


def parse_args(argv: Sequence[str] | None = None) -> argparse.Namespace:
    parser = argparse.ArgumentParser(
        prog="filereport",
        description="Walk directory trees and report file metadata.",
        epilog="Exit status: 0 ok, 1 partial failure, 2 usage error.",
    )
    parser.add_argument("roots", nargs="*", type=Path, default=
[Path.cwd()],
                        help="directories to scan (default: cwd)")
    parser.add_argument("-a", "--all", action="store_true",
                        help="include hidden names")
    parser.add_argument("-n", "--limit", type=int, metavar="COUNT",
                        help="stop after COUNT files")
    parser.add_argument("--sort", choices=("path", "size", "modified"),
```

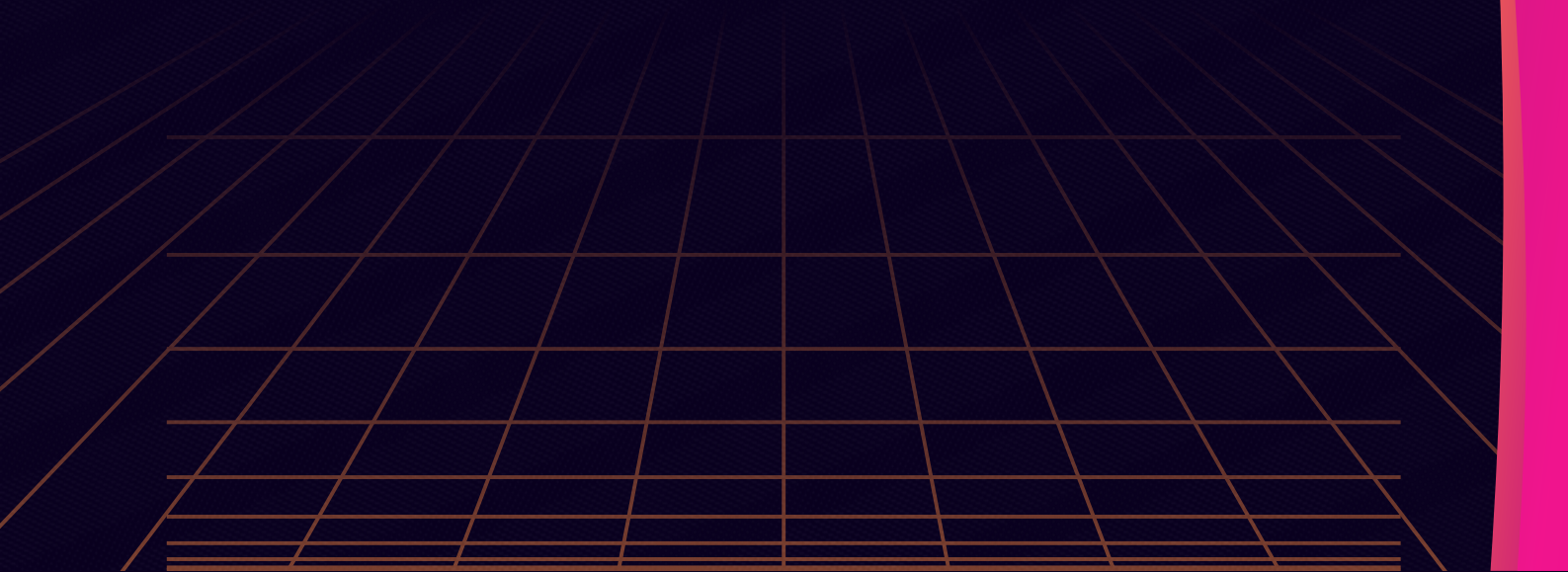
```
                        default="path", help="output order")
    parser.add_argument("--format", choices=("text", "json"),
default="text")
    parser.add_argument("--exclude", action="append", default=[],
                        metavar="GLOB", help="skip matching names
(repeatable)")
    parser.add_argument("-v", "--verbose", action="count", default=0,
                        help="-v for info, -vv for debug")
    parser.add_argument("--version", action="version", version="%(prog)s
1.0")
    return parser.parse_args(argv)


def main(argv: Sequence[str] | None = None) -> int:
    args = parse_args(argv)

    if args.limit is not None and args.limit < 0:
        print("--limit cannot be negative", file=sys.stderr)
        return 2

    ...
    return 0


if __name__ == "__main__":
    raise SystemExit(main())
```



YOU WANT	WRITE
A flag	`action="store_true"`
Countable verbosity	`action="count", default=0`
Repeatable option	`action="append", default=[]`
Fixed choices	`choices=(...)`
Converted value	`type=Path` , `type=int`
Optional positional	`nargs="?"`
Many positionals	`nargs="*"` or `"+"`
Mutually exclusive	`parser.add_mutually_exclusive_group()`
Subcommands	`parser.add_subparsers(dest="command", required=True)`

• TYPE= IS CONVERSION, NOT VALIDATION

`type=int` proves the text is an integer. It knows nothing about your domain rule that a limit cannot be negative. Validate after parsing and return exit status 2 – the Unix convention for usage errors.

• EXIT STATUSES, BY CONVENTION

`0` success · `1` the operation failed · `2` the caller used it wrong · `126/127` reserved by the shell · `130` killed by Ctrl-C. Scripts that always exit 0 are unusable in a pipeline.

33 THE TESTED MODULE

```
project/
├── filereport.py
└── test_filereport.py
```

```
import subprocess
import sys
import tempfile
import unittest
```

```
from pathlib import Path

from filereport import count_suffixes, main, walk_files


class CountSuffixesTests(unittest.TestCase):
    """Pure logic: no filesystem, no I/O, instant."""

    def test_counts_and_normalises_case(self):
        paths = [Path("a.PY"), Path("b.py"), Path("c.txt")]
        self.assertEqual(count_suffixes(paths)[".py"], 2)

    def test_extensionless(self):
        self.assertEqual(count_suffixes([Path("README")])["<none>"], 1)


class WalkFilesTests(unittest.TestCase):
    """Filesystem behaviour, in a directory the test owns."""

    def test_finds_nested_files(self):
        with tempfile.TemporaryDirectory() as raw:
            root = Path(raw)
            (root / "top.txt").write_text("x", encoding="utf-8")
            (root / "nested").mkdir()
            (root / "nested" / "deep.py").write_text("pass\n",
encoding="utf-8")

            found = {p.relative_to(root) for p in walk_files(root)}

        self.assertEqual(found, {Path("top.txt"), Path("nested/deep.py")})


class InterfaceTests(unittest.TestCase):
    """Two levels: the function, and the real process."""

    def test_main_returns_two_for_missing_root(self):
        self.assertEqual(main(["/definitely/not/here"]), 2)

    def test_process_boundary(self):
        result = subprocess.run(
            [sys.executable, "filereport.py", "--version"],
            text=True, capture_output=True, check=False,
        )
        self.assertEqual(result.returncode, 0)
```

```
if __name__ == "__main__":
    unittest.main()
```

```
python3 -m unittest -v          # standard library, zero install
python3 -m pytest -q            # nicer output, better assertions
```

• THE ORDER TO TEST IN

Pure logic first, filesystem second, the process last. Pure tests are milliseconds and catch most real bugs. Subprocess tests are slow and catch exactly one class of bug – that the thing actually runs – so you need very few of them.

34 THE INSTALLABLE PACKAGE

Promote when there is more than one module, or when the thing deserves to be a command on `PATH`.

```
filereport/
├── pyproject.toml
├── README.md
├── src/
│   └── filereport/
│       ├── __init__.py
│       ├── __main__.py      # enables: python -m filereport
│       ├── cli.py           # argparse, stdout/stderr, exit status
│       ├── scan.py          # traversal, filesystem
│       ├── model.py         # FileRecord
│       └── render.py        # text and JSON output
└── tests/
    ├── test_scan.py
    └── test_render.py
```

```toml
[build-system]
requires = ["setuptools>=77"]
build-backend = "setuptools.build_meta"

[project]
name = "filereport"
version = "0.1.0"
description = "Walk directory trees and report file metadata."
requires-python = ">=3.11"
dependencies = ["rich>=13"]

[project.optional-dependencies]
dev = ["pytest>=8", "mypy>=1.10", "ruff>=0.5"]

[project.scripts]
filereport = "filereport.cli:main"

[tool.ruff]
line-length = 100

[tool.mypy]
strict = true
```

```
python3 -m pip install -e ".[dev]"
filereport --help
python -m filereport --help
```

• DEPENDENCY DIRECTION

`cli` → `scan`, `render` → `model`. Nothing points back up. If splitting produces a circular import, the dependency direction is confused — move the shared model downward. Hiding an import inside a function to break a cycle is a patch over a design error.

• WHY SRC/

With a `src/` layout your tests cannot accidentally import the local folder instead of the installed package. That accident hides packaging bugs until someone else installs it.

35 THE LONG-RUNNING SERVICE

The last skeleton. Now signals, logging, config and clean shutdown all matter.

```
#!/usr/bin/env python3
"""A loop that survives contact with an init system."""

import logging
import os
import signal
import sys
import time
from types import FrameType

logger = logging.getLogger("watcher")
running = True


def handle_signal(signum: int, frame: FrameType | None) -> None:
    global running
    logger.info("received %s, shutting down", signal.Signals(signum).name)
    running = False


def configure_logging(verbosity: int = 0) -> None:
    level = {0: logging.WARNING, 1: logging.INFO}.get(verbosity,
logging.DEBUG)
    logging.basicConfig(
        level=level,
        format="%(asctime)s %(levelname)-8s %(name)s: %(message)s",
        stream=sys.stderr,
    )


def main() -> int:
    configure_logging(int(os.environ.get("VERBOSITY", "1")))
    signal.signal(signal.SIGTERM, handle_signal)
    signal.signal(signal.SIGINT, handle_signal)

    interval = float(os.environ.get("INTERVAL", "5"))
    logger.info("started, interval=%ss, pid=%s", interval, os.getpid())

    while running:
        try:
```

```
            do_one_pass()
        except Exception:
            logger.exception("pass failed; continuing")
        time.sleep(interval)

    logger.info("stopped cleanly")
    return 0


if __name__ == "__main__":
    raise SystemExit(main())
```

• WHAT MAKES THIS A SERVICE AND NOT A LOOP

- **SIGTERM is handled** – systemd and Docker send it first, then `SIGKILL`. Ignore it and you get killed mid-write.
- **Logs go to stderr**, timestamped, with levels. The init system captures them.
- **Config from the environment**, so nothing is baked into the image.
- `logger.exception` **inside the loop** – one failed pass records a full traceback and does not kill the process.

This is the one place a broad `except Exception` is correct – because it logs the traceback and the loop is explicitly designed to survive.

• CHALLENGE LEVEL 4

Take the level-one script at the top of this part and walk it all the way to the installable package, in five commits: functions, then `main()`, then argparse, then tests, then `pyproject.toml`. Behaviour must be identical at every step.

Hint: write the tests *before* the argparse step; they are what prove behaviour did not change. If a refactor breaks a test you had already written, that is the exercise working.

PROGRESS – PART VI

- [ ] Typed the `main()` skeleton from memory
- [ ] Built a CLI with a flag, a choice and a repeatable option
- [ ] Returned three distinct exit statuses on purpose
- [ ] Wrote pure, filesystem and subprocess tests
- [ ] Installed something of mine with `pip install -e`
- [ ] Handled SIGTERM in a loop and shut down cleanly
- [ ] Completed the five-commit promotion

PART VII

VII IMPORTS

What an import actually does, the standard library that covers most of your work, and how to decide whether a third-party package deserves to be in your dependency list.

36 WHAT EACH FORM BINDS

FORM	BINDS	BEST FOR
`import json`	The module object	Provenance; large APIs
`from pathlib import Path`	One member	Names used constantly
`import numpy as np`	The module, renamed	Established conventions only
`from x import y as z`	One member, renamed	Real collisions
`from x import *`	Everything public	Nothing. Ever.

• WHY NOT WILDCARD

You lose the visible inventory of what is bound, you invite silent collisions, and a reader has to search elsewhere to learn where a name came from.

37 MODULES, PACKAGES, AND CACHING

• THE ONE SENTENCE

On first import Python finds the module, creates a module object, executes its top-level code once, and caches it in `sys.modules`. Every later import reuses that object.

```
import sys, json
"json" in sys.modules       # True
json.__file__               # where it came from
json.__name__               # 'json'
```

That is why a module should define functions, classes and constants – not scan disks, prompt, or make network calls merely because someone imported it.

```
from .scan import walk_files          # relative: from this package
from filereport.scan import walk_files  # absolute: clearer in applications
```

• TRAP – SHADOWING THE STANDARD LIBRARY

Never name a file `json.py`, `random.py`, `types.py`, `email.py`, `select.py`, `datetime.py` or `pathlib.py`. Your file wins, the real module vanishes, and the error message names a line you did not write. Delete the stray `__pycache__` too.

38 THE STANDARD LIBRARY YOU WILL ACTUALLY USE

Installed everywhere, versioned with the interpreter, no supply chain. Reach here first, every time.

MODULE	FOR
`pathlib`	Paths as objects. Replaces most of `os.path`
`os` / `os.walk`	Environment, process, prunable tree walks
`shutil`	Copy, move, rmtree, disk usage, `which`
`sys`	`argv`, `stderr`, `exit`, `executable`, `modules`
`argparse`	Command-line interfaces
`subprocess`	Running other programs
`logging`	Levelled, timestamped diagnostics
`json`	JSON in and out
`csv`	Delimited files, quoting handled properly

MODULE	FOR
`tomllib`	Reading TOML config (3.11+, read-only)
`configparser`	INI-style config
`sqlite3`	A real database, zero setup
`datetime` / `zoneinfo`	Time, and time zones done correctly
`re`	Regular expressions
`collections`	`Counter`, `defaultdict`, `deque`, `namedtuple`
`itertools` / `functools`	Lazy combinators; caching, `wraps`, `partial`
`dataclasses`	Record types
`enum`	Named constant sets
`typing` / `collections.abc`	Annotations and abstract types
`contextlib`	`contextmanager`, `suppress`, `ExitStack`
`tempfile`	Temporary files and directories that clean up
`hashlib` / `secrets`	Digests; cryptographically safe randomness
`unittest`	Tests with no install required
`urllib.parse`	Splitting and building URLs correctly
`http.server`	A throwaway static server. Never in production
`concurrent.futures`	Thread and process pools with one API
`signal`	SIGTERM, SIGINT handling
`textwrap` / `difflib` / `string`	Wrapping, diffs, templates

• TWO THAT SURPRISE PEOPLE

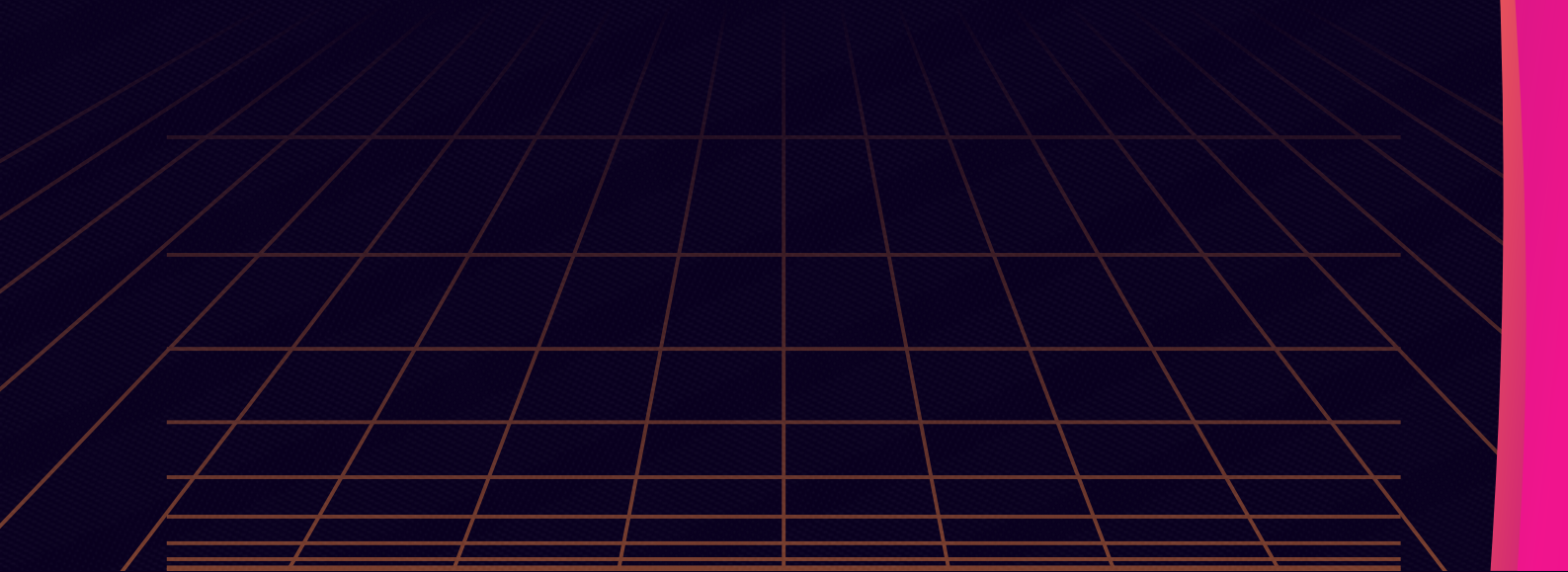
`sqlite3` is a complete transactional database in the standard library – for anything local, it beats inventing a file format. `zoneinfo` means you no longer need a third-party package to handle time zones correctly.

39 VETTED THIRD PARTY, AND HOW TO JUDGE ONE

• THE FIVE-QUESTION TEST

1. **Can the standard library do it?** If yes, stop. A dependency is permanent.
2. **Is it still maintained?** A release in the last year, and issues that get answered.
3. **How many dependencies does *it* pull in?** You are adopting the whole tree.
4. **Who else relies on it?** Download counts lie a little; being a dependency of something you already trust lies less.
5. **What happens if it is abandoned?** Could you replace it in a day, or is it load-bearing in every module?

Pin versions, review the lockfile, and never install from a name someone pasted you without reading it – typosquatting on PyPI is real.



39.1 The short list that has earned it

PACKAGE	FOR	INSTEAD OF
`requests`	HTTP the readable way	`urllib.request`
`httpx`	Same API, plus async and HTTP/2	`requests`, when you need either
`rich`	Tables, colour, progress bars, pretty tracebacks	Hand-rolled ANSI codes
`typer` / `click`	CLIs from type hints or decorators	`argparse`, on bigger tools
`pydantic`	Validated, parsed, typed data at a boundary	Hand-written dict checking
`pytest`	Plain `assert`, fixtures, parametrising	`unittest`, as suites grow
`ruff`	Lint and format, extremely fast	flake8 + isort + black
`mypy`	Static type checking	Hoping
`fastapi`	Typed HTTP APIs with generated docs	Flask, for new API work
`pandas`	Tabular data, joins, group-bys	Nested dicts and loops
`polars`	The same, faster, stricter	`pandas`, on large data
`matplotlib`	Charts, including to file	Exporting to a spreadsheet
`paramiko` / `fabric`	SSH from Python	Shelling out to `ssh`
`psutil`	Processes, CPU, memory, disks, cross-platform	Parsing `ps` output
`uv` / `pipx`	Fast installs; isolated CLI tools	A polluted global environment

• ENVIRONMENTS, BRIEFLY, BECAUSE IT BITES EVERYONE

```
python3 -m venv .venv && source .venv/bin/activate
python -m pip install -r requirements.txt
pip freeze > requirements.txt        # or use uv / pip-tools for
real pinning
```

Always `python -m pip`, never bare `pip` — it guarantees you are installing into the interpreter you think you are. Never `sudo pip install` into the system Python; that is how you break your operating system's own tooling.

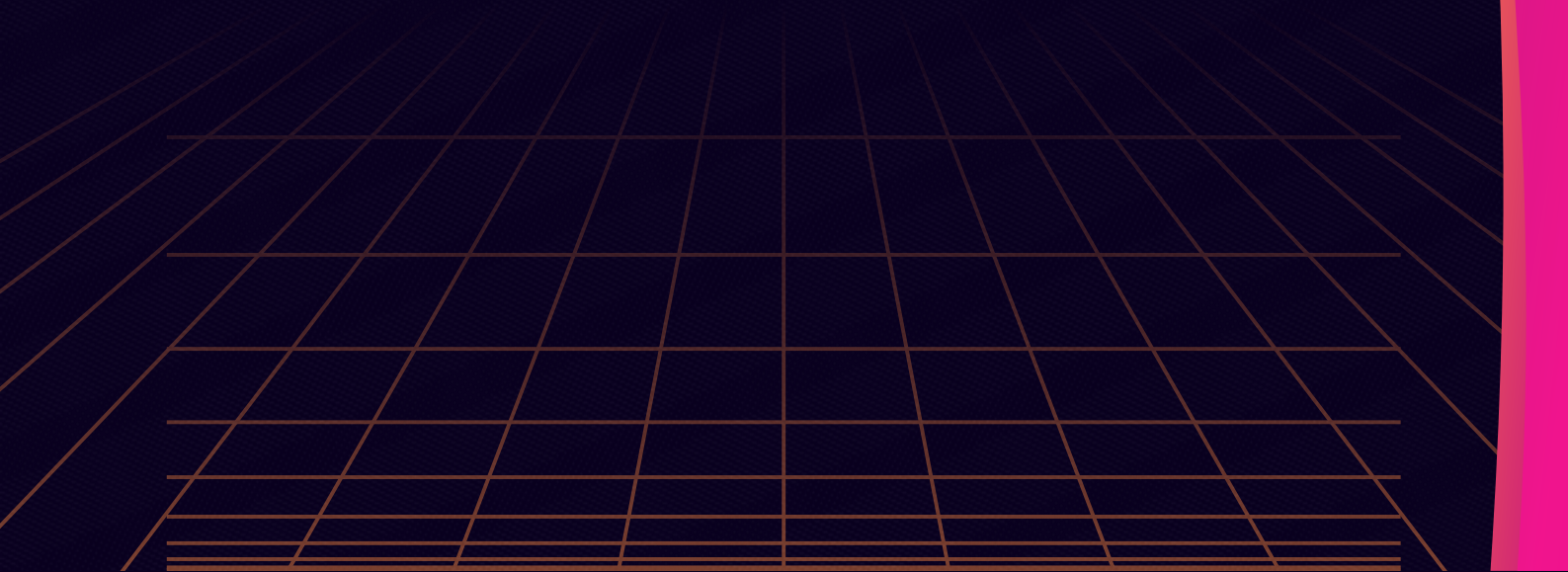
• CHALLENGE LEVEL 2

Take any script of yours that imports a third-party package and rewrite it using only the standard library. Then decide, with reasons, whether the dependency was worth it. Write the two-sentence justification you would put in a pull request.

Hint: the answer is genuinely sometimes "yes, keep it." `requests` over raw `urllib.request` for anything with retries, sessions or auth is usually defensible. The exercise is being able to say *why*.

PROGRESS — PART VII

- [ ] Can state what `import` does in one sentence
- [ ] Shadowed a stdlib module on purpose and recovered
- [ ] Replaced a third-party call with a standard-library one
- [ ] Applied the five-question test to a real dependency
- [ ] Built a project in a venv with pinned versions



PART VIII

VIII

PUZZLE PIECES

Working code, every line explained, sized to drop into the skeletons from Part VI. Take a piece, put it inside `main()`, wire it to an argparse flag. That is the whole method.

• HOW TO USE A PIECE

Each one is a function with a stated contract and no global state, so it composes. The annotation under each piece explains the lines that are not obvious — not the ones that are. Type the piece, run it, break one line, then fit it to your own problem.

40 FILES AND DIRECTORIES

40.1 Walk a tree, prune subdirectories, stay lazy

```
import os
from collections.abc import Iterator
from pathlib import Path

SKIP = {".git", "node_modules", "__pycache__", ".venv"}

def walk_files(root: Path, *, include_hidden: bool = False) ->
Iterator[Path]:
    for current, dirnames, filenames in os.walk(root, onerror=None):
        dirnames[:] = [d for d in dirnames
                       if d not in SKIP and (include_hidden or not
d.startswith("."))]
        for name in filenames:
            if include_hidden or not name.startswith("."):
                yield Path(current) / name
```

dirnames[:] = [...]

> Slice assignment mutates the list `os.walk` is holding, so it never descends into the pruned directories. Rebinding with `dirnames = [...]` would create a new list and prune nothing – the walk continues into every folder and you have merely wasted a line.

keyword-only include_hidden

> The `*` forbids `walk_files(root, True)`, which nobody can read at the call site.

yield

> Nothing is collected. A caller can `break` after ten files and the walk stops there.

40.2 Safe atomic write

```
import os, tempfile
from pathlib import Path

def write_atomic(path: Path, text: str, *, encoding: str = "utf-8") ->
None:
    directory = path.parent
    directory.mkdir(parents=True, exist_ok=True)
    fd, temp_name = tempfile.mkstemp(dir=directory, prefix=f".
{path.name}.")
    try:
        with os.fdopen(fd, "w", encoding=encoding) as handle:
            handle.write(text)
            handle.flush()
            os.fsync(handle.fileno())
        os.replace(temp_name, path)
    except BaseException:
        Path(temp_name).unlink(missing_ok=True)
        raise
```

mkstemp(dir=directory)

> The temporary file must be on the same filesystem as the target, or the final rename stops being atomic and becomes a copy.

flush() then fsync()

> `flush` pushes Python's buffer to the OS; `fsync` pushes the OS cache to the disk. Without both, a power cut can leave a file that exists and is empty.

os.replace

> Atomic on POSIX and Windows. A reader sees either the entire old file or the entire new one — never a half-written one.

except BaseException

> Deliberately broader than `Exception`, so that Ctrl-C also cleans up the temp file. It re-raises immediately, so it hides nothing.

40.3 Find duplicate files by content

```
import hashlib
from collections import defaultdict

def digest(path: Path, chunk: int = 1 << 20) -> str:
    h = hashlib.blake2b(digest_size=16)
    with path.open("rb") as handle:
        while block := handle.read(chunk):
            h.update(block)
    return h.hexdigest()

def duplicates(paths: Iterable[Path]) -> dict[str, list[Path]]:
    by_size: dict[int, list[Path]] = defaultdict(list)
    for path in paths:
        try:
            by_size[path.stat().st_size].append(path)
        except OSError:
            continue

    found: dict[str, list[Path]] = defaultdict(list)
    for same_size in by_size.values():
        if len(same_size) < 2:
            continue                        # unique size cannot be a
duplicate
        for path in same_size:
            found[digest(path)].append(path)
    return {d: ps for d, ps in found.items() if len(ps) > 1}
```

Size first, hash second

> Hashing every file is expensive. Files with a unique size cannot possibly be duplicates, so this typically avoids hashing 90% of the tree. That check is the whole algorithm.

while block := handle.read(chunk)

> The walrus reads and tests in one expression, so a 4 GB file never enters memory. The loop ends when `read` returns an empty `bytes`, which is falsy.

blake2b over md5

> Faster than MD5 and not broken. `digest_size=16` keeps the output short; collisions are not a practical concern here.

41 TEXT AND LOG PROCESSING

41.1 Stream lines from anything, including stdin and .gz

```
import gzip, sys
from collections.abc import Iterator
from pathlib import Path

def read_lines(source: Path | None) -> Iterator[str]:
    if source is None:
        yield from (line.rstrip("\n") for line in sys.stdin)
        return
    opener = gzip.open if source.suffix == ".gz" else open
    with opener(source, "rt", encoding="utf-8", errors="replace") as
handle:
        for line in handle:
            yield line.rstrip("\n")
```

source: Path | None

> `None` means stdin. That single convention makes the tool pipeable, which is most of what makes a Unix tool useful.

opener = gzip.open if ...

> Both functions have a compatible signature, so binding one to a name and calling it later avoids duplicating the whole `with` block.

"rt" and errors="replace"

> Text mode, and one bad byte in a 10 GB log becomes a replacement character instead of killing the run at 3 a.m.

41.2 Parse structured log lines into records

```
import re
from datetime import datetime

LINE = re.compile(r"""
    ^(?P<stamp>\d{4}-\d{2}-\d{2}[ T]\d{2}:\d{2}:\d{2})
    \s+(?P<level>DEBUG|INFO|WARNING|ERROR|CRITICAL)
    \s+(?P<logger>[\w.]+)
    :\s*(?P<message>.*)$
""", re.VERBOSE)

def parse(lines: Iterable[str]) -> Iterator[dict[str, object]]:
    for number, line in enumerate(lines, start=1):
        m = LINE.match(line)
        if m is None:
            continue                        # decide: skip, or count, or
raise
        row = m.groupdict()
        row["stamp"] = datetime.fromisoformat(row["stamp"])
        row["line_number"] = number
        yield row
```

Compiled at module level

> Compiled once, not once per line. On a million-line file that is a measurable difference.

re.VERBOSE

> Lets the pattern span lines with comments. A one-line version of this regex is unmaintainable within a week.

enumerate(lines, start=1)

> Humans and editors number lines from 1. Reporting a 0-indexed line number to a person is a small, constant irritation.

if m is None: continue

> A visible policy decision. Silently skipping unmatched lines can hide a pattern that broke after a log-format change — counting them and reporting the total at the end is usually better.

41.3 Count, group, and report

```
from collections import Counter, defaultdict

def summarise(rows: Iterable[dict]) -> tuple[Counter, dict[str, Counter]]:
    levels = Counter()
    per_logger: dict[str, Counter] = defaultdict(Counter)
    for row in rows:
        levels[row["level"]] += 1
        per_logger[row["logger"]][row["level"]] += 1
    return levels, per_logger

def report(levels: Counter, width: int = 40) -> Iterator[str]:
    total = sum(levels.values()) or 1
    for level, count in levels.most_common():
        bar = "█" * round(width * count / total)
        yield f"{level:<9} {count:>8,}  {count / total:>6.1%}  {bar}"
```

defaultdict(Counter)

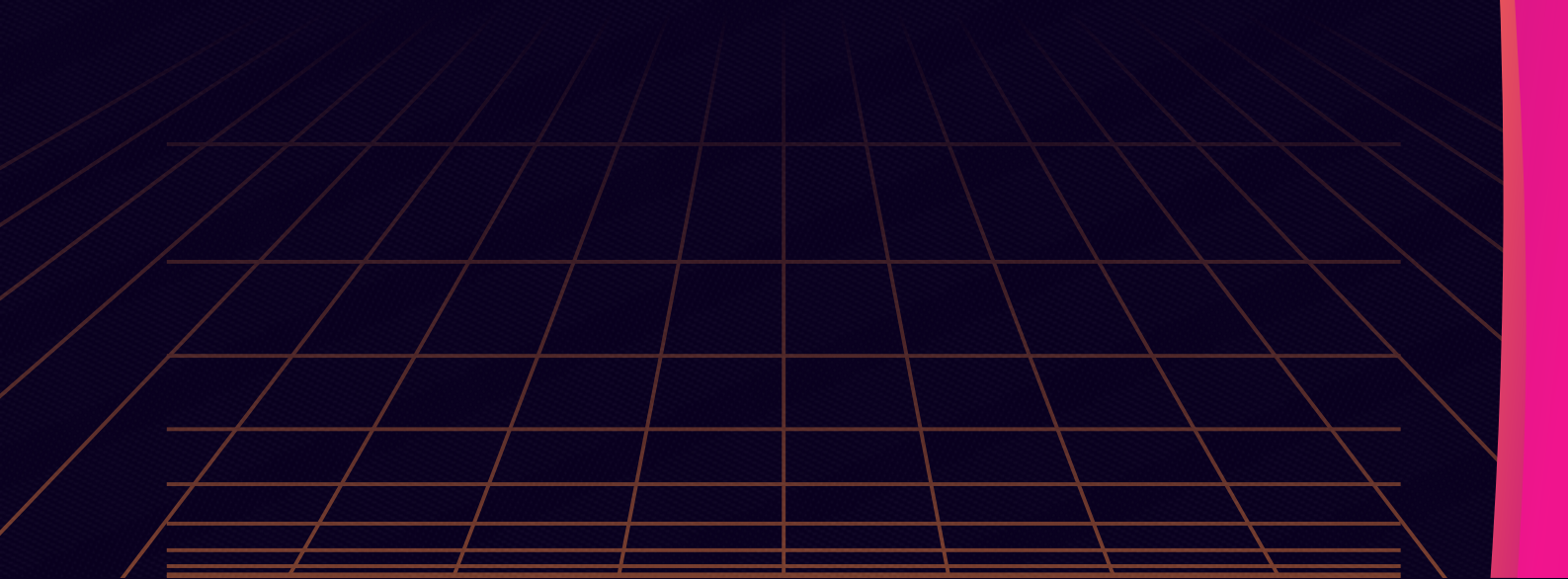
> A missing logger key creates an empty `Counter` automatically, so there is no "if key not in" branch anywhere.

or 1

> Guards against division by zero on empty input. Cheap, and it stops the whole report crashing on a file with no matches.

yield, not print

> The caller decides where lines go – stdout, a file, a test assertion. A function that prints cannot be tested.



42 SYSADMIN AND THE ENVIRONMENT

42.1 Configuration with a clear precedence order

```
import os, tomllib
from dataclasses import dataclass
from pathlib import Path

@dataclass(frozen=True, slots=True)
class Config:
    interval: float = 5.0
    root: Path = Path.cwd()
    verbose: bool = False

def load_config(path: Path | None, overrides: dict[str, object]) ->
Config:
    values: dict[str, object] = {}

    if path and path.is_file():                         # 1. file
        values |= tomllib.loads(path.read_text("utf-8"))

    for key in ("interval", "root", "verbose"):         # 2. environment
        env = os.environ.get(f"APP_{key.upper()}")
        if env is not None:
            values[key] = env

    values |= {k: v for k, v in overrides.items() if v is not None}   # 3.
CLI

    return Config(
        interval=float(values.get("interval", 5.0)),
        root=Path(values.get("root", Path.cwd())).expanduser(),
        verbose=str(values.get("verbose", "")).lower() in {"1", "true",
"yes"},
    )
```

File, then environment, then CLI

> The universal precedence order: the most specific source wins. A user passing a flag always beats a config file they forgot about.

if v is not None

> argparse fills unsupplied options with `None`. Without this filter, every unset flag would overwrite the config file with nothing.

Conversion at the boundary

> Environment variables are always strings. `"false"` is a truthy string — converting at the edge means the rest of the program handles real types and never repeats this check.

tomllib

> Standard library since 3.11, read-only. No dependency needed for config.

42.2 Disk usage and a low-space guard

```
import shutil

def disk_report(path: Path) -> str:
    usage = shutil.disk_usage(path)
    used_pct = usage.used / usage.total
    return (f"{path}: {usage.free / 1e9:.1f} GB free of "
            f"{usage.total / 1e9:.1f} GB ({used_pct:.0%} used)")

def require_free_space(path: Path, need_bytes: int) -> None:
    free = shutil.disk_usage(path).free
    if free < need_bytes:
        raise OSError(f"need {need_bytes:,} bytes at {path}, only {free:,}
free")
```

shutil.disk_usage

> Cross-platform, one call, returns a named tuple of total/used/free. No parsing of `df` output, which differs between systems and locales.

Check before writing

> Failing early with a clear message beats failing half-way through a 5 GB export with `ENOSPC` and a corrupt output file.

42.3 A lock file so two copies never run at once

```
import fcntl, os
from contextlib import contextmanager

@contextmanager
def single_instance(lock_path: Path):
    lock_path.parent.mkdir(parents=True, exist_ok=True)
    handle = lock_path.open("w")
    try:
        fcntl.flock(handle, fcntl.LOCK_EX | fcntl.LOCK_NB)
    except BlockingIOError:
        handle.close()
        raise SystemExit(f"already running (lock: {lock_path})")
    try:
        handle.write(str(os.getpid()))
        handle.flush()
        yield
    finally:
        fcntl.flock(handle, fcntl.LOCK_UN)
        handle.close()

with single_instance(Path("/tmp/myjob.lock")):
    do_work()
```

LOCK_NB

> Non-blocking: fail immediately rather than queueing. For a cron job that runs every minute, queueing is how you end up with forty copies.

flock, not a PID file

> The kernel releases the lock when the process dies, even on `SIGKILL`. A plain PID file survives a crash and blocks every future run.

POSIX only

> `fcntl` does not exist on Windows. For cross-platform, `msvcrt.locking` or a small third-party package is needed.

43 HTTP: CALLING AN API

43.1 A session with retries, timeouts and pagination

```
import requests
from requests.adapters import HTTPAdapter
from urllib3.util.retry import Retry

def make_session(token: str | None = None, retries: int = 3) ->
requests.Session:
    session = requests.Session()
    policy = Retry(
        total=retries,
        backoff_factor=0.5,
        status_forcelist=(429, 500, 502, 503, 504),
        allowed_methods=frozenset({"GET", "HEAD", "OPTIONS"}),
        respect_retry_after_header=True,
    )
    session.mount("https://", HTTPAdapter(max_retries=policy))
    session.headers.update({"User-Agent": "filereport/1.0"})
    if token:
        session.headers["Authorization"] = f"Bearer {token}"
    return session

def paginate(session, url: str, *, timeout: float = 10.0) ->
Iterator[dict]:
    while url:
        response = session.get(url, timeout=timeout)
        response.raise_for_status()
        payload = response.json()
        yield from payload["results"]
        url = payload.get("next")
```

A Session, not bare requests.get

> Reuses the TCP connection and the TLS handshake across calls, and carries headers once. On a hundred requests this is the difference between seconds and minutes.

backoff_factor=0.5

> Exponential waits: 0.5 s, 1 s, 2 s. Hammering a failing server immediately is how you get rate-limited or blocked.

allowed_methods

> Only retry idempotent verbs. Retrying a `POST` can create the same record twice.

timeout= on every call

> `Requests has no default timeout.` Omit it and a hung server hangs your program forever. This is the single most common production bug with this library.

yield from + next

> The caller iterates records and never sees pages at all. Memory stays flat regardless of how many pages exist.

43.2 Handle the response honestly

```
import json, logging
logger = logging.getLogger(__name__)

def fetch_json(session, url: str, **kwargs) -> dict | None:
    try:
        response = session.get(url, timeout=kwargs.pop("timeout", 10))
        response.raise_for_status()
        return response.json()
    except requests.Timeout:
        logger.warning("timeout: %s", url)
    except requests.HTTPError as error:
        logger.warning("http %s: %s", error.response.status_code, url)
    except requests.RequestException as error:
        logger.warning("request failed: %s (%s)", url, error)
    except json.JSONDecodeError:
        logger.warning("not JSON: %s", url)
    return None
```

raise_for_status()

> Without it, a 404 is a perfectly successful response object containing an error page, and `.json()` fails later with a confusing message.

Four separate excepts

> Each failure gets a distinct log line, so the diagnosis is in the log rather than in your head. `RequestException` is the base class, so it goes last.

Returning None

An explicit contract: the caller must handle absence. For a batch job that must not die on one bad URL, this is right; for a single interactive request, raising is better.

44 HTTP: SERVING AN API

44.1 A typed, validated endpoint

```
from fastapi import FastAPI, HTTPException, Query
from pydantic import BaseModel, Field

app = FastAPI(title="File report", version="1.0")

class FileOut(BaseModel):
    path: str
    size: int = Field(ge=0)
    modified: datetime

class ScanRequest(BaseModel):
    root: str
    include_hidden: bool = False
    limit: int = Field(default=100, ge=1, le=10_000)

@app.get("/health")
def health() -> dict[str, str]:
    return {"status": "ok"}

@app.post("/scan", response_model=list[FileOut])
def scan(request: ScanRequest) -> list[FileOut]:
    root = Path(request.root)
    if not root.is_dir():
        raise HTTPException(status_code=404, detail=f"not a directory:
{root}")
    records = islice(collect(walk_files(root,
                     include_hidden=request.include_hidden)),
request.limit)
    return [FileOut(path=str(r.path), size=r.size, modified=r.modified)
            for r in records]
```

BaseModel

Pydantic parses and validates the incoming JSON before your function runs. A missing field or a string where an int belongs returns a 422 with a precise message, and you wrote no validation code.

Field(ge=1, le=10_000)

The domain rule lives in the schema, so it is enforced and documented in one place. Compare this with the manual negative-limit check in the argparse skeleton.

response_model

Filters the output to exactly these fields. It is how you avoid leaking an internal attribute you added six months later.

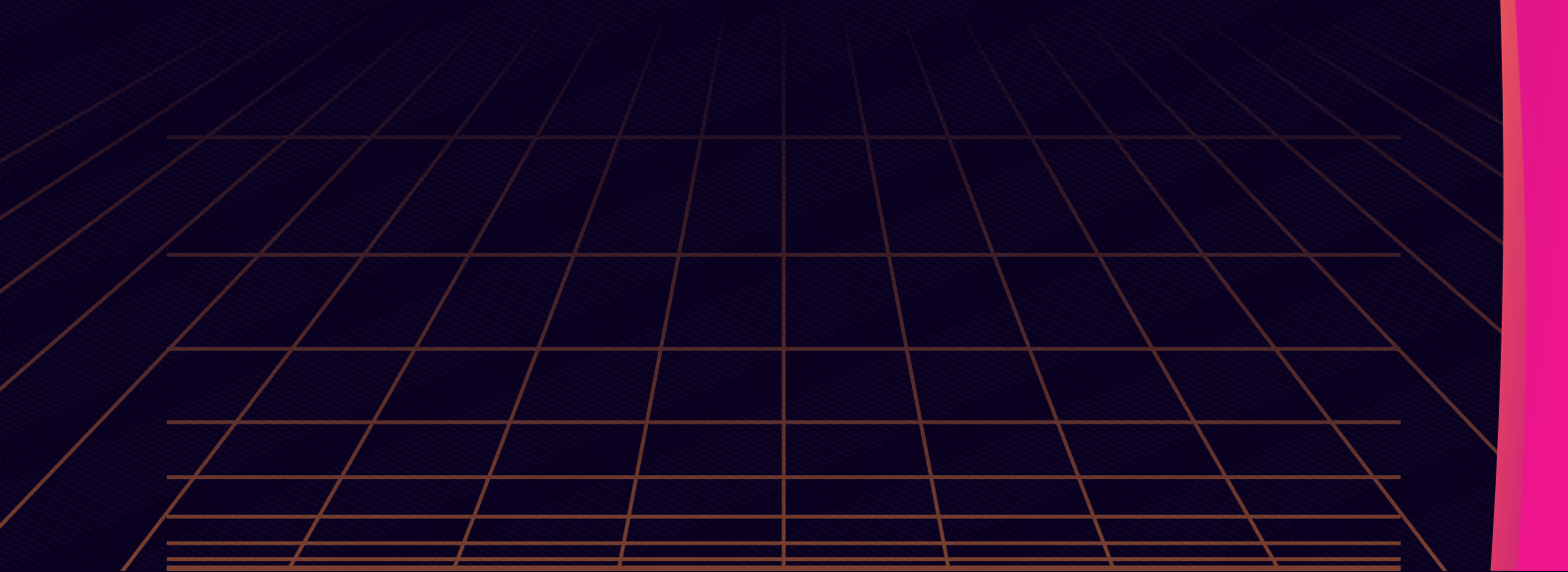
HTTPException

Status code plus detail, serialised correctly. Never `return {"error": ...}` with a 200.

/health

Every deployed service needs one. Load balancers and orchestrators require something cheap to poll that touches no database.

```
python -m pip install "fastapi[standard]"
fastapi dev main.py          # docs at /docs, schema at /openapi.json
```



45 DATA AND CHARTS

45.1 Load, clean, group, and export

```
import pandas as pd

def load(path: Path) -> pd.DataFrame:
    frame = pd.read_csv(
        path,
        parse_dates=["timestamp"],
        dtype={"level": "category", "logger": "category"},
        na_values=["", "-", "NULL"],
    )
    frame = frame.dropna(subset=["timestamp", "level"])
    frame["hour"] = frame["timestamp"].dt.floor("h")
    return frame

def errors_per_hour(frame: pd.DataFrame) -> pd.DataFrame:
    errors = frame[frame["level"].isin(["ERROR", "CRITICAL"])]
    return (errors
            .groupby(["hour", "logger"], observed=True)
            .size()
            .reset_index(name="count")
            .sort_values("count", ascending=False))
```

dtype="category"

> A column with few distinct values stored as categories uses a fraction of the memory and groups far faster. On log data this is often a 10× difference.

na_values

> Real data encodes “missing” as `-`, empty, or `NULL`. Declaring them here means `dropna` works instead of silently keeping the string `"-"`.

.dt.floor("h")

> Truncates each timestamp to its hour — the standard way to build a time bucket for grouping.

observed=True

> With categorical keys, pandas otherwise emits a row for every unused combination. On two categorical columns that can be a million empty rows.

Method chaining in parentheses

> Each step is one readable line, and any line can be commented out to inspect the intermediate result.

45.2 A chart written to a file

```
import matplotlib
matplotlib.use("Agg")                    # before pyplot: no display needed
import matplotlib.pyplot as plt

def plot_hourly(frame: pd.DataFrame, out: Path) -> Path:
    series = frame.groupby("hour", observed=True).size()

    fig, ax = plt.subplots(figsize=(10, 4), dpi=150)
    ax.plot(series.index, series.values, linewidth=1.5)
    ax.set_title("Errors per hour")
    ax.set_xlabel("")
    ax.set_ylabel("count")
    ax.grid(True, alpha=0.3)
    fig.autofmt_xdate()
    fig.tight_layout()
    fig.savefig(out)
    plt.close(fig)
    return out
```

matplotlib.use("Agg")

> Selects the file-only backend. Without it, matplotlib tries to find a display and crashes on a server or in CI. It must come before importing `pyplot`.

fig, ax = plt.subplots()

> The object API. The alternative `plt.plot()` style relies on hidden global state and breaks the moment you draw two charts.

plt.close(fig)

> Figures are not garbage collected while pyplot holds a reference. In a loop generating 500 charts, omitting this exhausts memory.

Returning the path

> Makes the function testable — assert the file exists and is non-empty — and composable into a report pipeline.

46 ORIENTATION: DESKTOP GUI

Not taught here, but you should know the shape of the decision.

OPTION	REACH FOR IT WHEN	COST
`tkinter`	Standard library. An internal tool, a few widgets, no install	Looks dated; layout is fiddly
`PySide6` / Qt	A real desktop application, native feel, rich widgets	Large dependency; LGPL terms to read
`textual`	A terminal UI — often the right answer for a sysadmin tool	Terminal only
A local web UI	You already know HTML; users have browsers	Two languages, a server to run

• THE SHAPE EVERY GUI SHARES

An event loop owns the main thread. Your code responds to callbacks. Blocking that thread — a slow file scan, an HTTP call — freezes the window. Long work goes on a worker thread and reports back via the toolkit's own queue or signal mechanism, never by touching a widget directly from another thread. Given Part IX, you already know how to do the threading part.

47 ORIENTATION: MACHINE LEARNING

• WHERE IT ACTUALLY STARTS

Not with a neural network. The path is: `numpy` for arrays → `pandas` for tables → `scikit-learn` for classical models, which covers most real problems → `pytorch` only when the data is images, audio or language. Every step assumes the one before it.

```
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import classification_report

X_train, X_test, y_train, y_test = train_test_split(
    features, labels, test_size=0.2, random_state=0, stratify=labels)

model = RandomForestClassifier(n_estimators=200, random_state=0)
model.fit(X_train, y_train)
print(classification_report(y_test, model.predict(X_test)))
```

train_test_split

> Evaluating on data the model trained on tells you nothing. This is the first rule and the most commonly broken one.

stratify=labels

> Keeps the class balance the same in both halves. Without it, a rare class can be absent from the test set entirely.

random_state=0

> Reproducibility. Without a fixed seed you cannot tell an improvement from noise.

classification_report

> Precision, recall and F1 per class. Accuracy alone is meaningless on imbalanced data — 99% accuracy is trivial when 99% of rows are one class.

• THE HONEST WARNING

This is four lines of API over a large amount of statistics. The code is never the hard part; the hard parts are the data, the leakage, and the evaluation. Treat this page as a map, not a lesson.

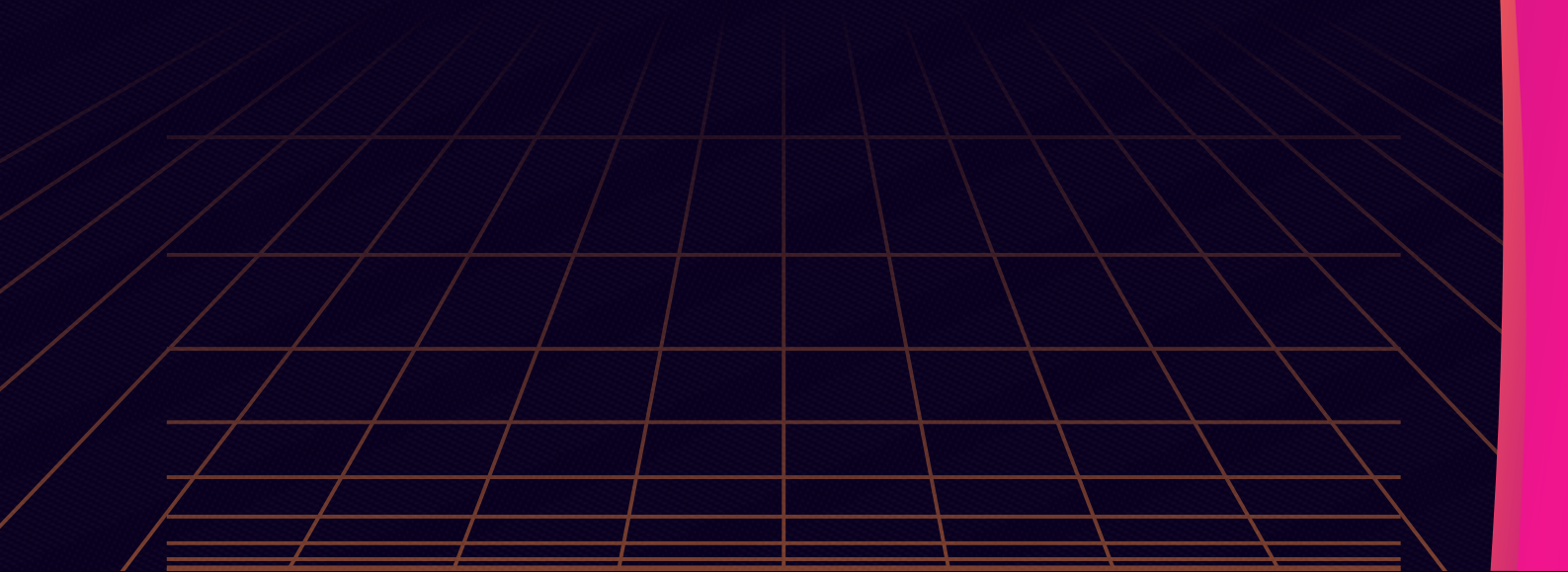
• CHALLENGE LEVEL 4

Combine four pieces into one tool: stream a `.gz` log from stdin or a path, parse it with the named-group regex, summarise per level and logger, and write both a text report to stdout and a PNG chart to a file. Drive all of it from the argparse skeleton, with `--chart PATH` optional and the whole thing pipeable.

Hint: the pieces already fit — the work is the seams. Decide up front which stage owns failure: what happens when zero lines match, when the output directory does not exist, and when stdout is closed early by `| head`.

PROGRESS — PART VIII

- [ ] Used the pruning walk on a real tree
- [ ] Wrote a file atomically and verified with a forced crash
- [ ] Built the size-then-hash duplicate finder
- [ ] Made a tool that reads from stdin as well as a path
- [ ] Parsed a log with named groups into typed records
- [ ] Added retries and a timeout to every outbound request
- [ ] Served a validated endpoint and read its generated docs
- [ ] Wrote a chart to a file from a headless process
- [ ] Completed the four-piece combination challenge



PART IX

IX

PROCESSES AND CONCURRENCY

Running other programs, and running your own code more than once at a time. The rule that governs all of it: measure first, and keep the boring version as the reference.

48 SUBPROCESS, CORRECTLY

```
import subprocess, sys

result = subprocess.run(
    ["git", "rev-parse", "--short", "HEAD"],
    capture_output=True,
    text=True,
    timeout=10,
    check=False,
    cwd=repo,
)
if result.returncode != 0:
    print(result.stderr.strip(), file=sys.stderr)
    raise SystemExit(result.returncode)

commit = result.stdout.strip()
```

ARGUMENT	WHY IT IS THERE
A **list**, not a string	No shell parsing, so spaces and quotes in filenames are safe
`capture_output=True`	Fills `.stdout` and `.stderr` instead of letting them leak to your terminal
`text=True`	`str` instead of `bytes`, decoded with the locale encoding
`timeout=10`	A hung child otherwise hangs you forever
`check=False`	You handle the status; `check=True` raises `CalledProcessError` instead
`cwd=`	Set the working directory rather than `os.chdir`, which is global and not thread-safe

• NEVER BUILD A SHELL STRING

```
subprocess.run(f"rm -rf {user_input}", shell=True)   # command injection
```

`shell=False` is the default; keep it. If you genuinely need a pipeline, build it with two `Popen` calls or – better – do the second half in Python. You are already holding the data.

48.1 Streaming a child's output as it arrives

```
with subprocess.Popen(
    ["rsync", "-a", "--info=progress2", str(src), str(dst)],
    stdout=subprocess.PIPE, stderr=subprocess.STDOUT,
    text=True, bufsize=1,
) as proc:
    for line in proc.stdout:
        logger.info("rsync: %s", line.rstrip())

if proc.returncode != 0:
    raise RuntimeError(f"rsync failed: {proc.returncode}")
```

Popen over run

`run` waits for completion and hands you everything at once. `Popen` lets you consume output live – essential for anything long-running.

stderr=subprocess.STDOUT

> Merges the two streams so interleaved progress and errors stay in order.

The with block

> Waits for the child and closes the pipes. Without it, a forgotten process becomes a zombie.

49 THREADS FOR I/O

• THE ONE RULE

Threads for waiting. Processes for computing. The GIL means only one thread runs Python bytecode at a time – but a thread blocked on a socket or a disk read is not running bytecode, so it releases it. That is why threads help network and file work and do nothing for arithmetic.

```
from concurrent.futures import ThreadPoolExecutor, as_completed

def fetch_all(session, urls: list[str], workers: int = 8) -> dict[str,
dict | None]:
    results: dict[str, dict | None] = {}
    with ThreadPoolExecutor(max_workers=workers) as pool:
        futures = {pool.submit(fetch_json, session, url): url for url in
urls}
        for future in as_completed(futures):
            url = futures[future]
            try:
                results[url] = future.result()
            except Exception:
                logger.exception("failed: %s", url)
                results[url] = None
    return results
```

The dict comprehension of futures

> Maps each future back to its URL. `as_completed` yields in finishing order, not submission order, so without this you cannot tell which result is which.

future.result() inside try

> An exception in a worker is stored, not raised, until you call `result()`. Never calling it means the failure vanishes silently.

with ... as pool

Shuts down the pool and waits for the workers. Otherwise the interpreter can exit with work in flight.

max_workers=8

> More is not better. Past a point you are just queueing, and a polite client does not open two hundred connections to one host.

49.1 Shared state needs a lock

```
import threading

class Counter:
    def __init__(self) -> None:
        self._lock = threading.Lock()
        self._n = 0

    def increment(self) -> None:
        with self._lock:
            self._n += 1        # read, add, write — three steps,
interruptible

    @property
    def value(self) -> int:
        with self._lock:
            return self._n
```

• BETTER: DO NOT SHARE

Have each worker return a value and combine the results in the main thread — exactly what the `fetch_all` piece does. A design with no shared mutable state cannot have a race condition. Reach for a lock only when that is genuinely impossible, and use `queue.Queue` when threads must hand work to each other.

50 PROCESSES FOR CPU

```
from concurrent.futures import ProcessPoolExecutor

def hash_all(paths: list[Path], workers: int | None = None) -> dict[Path,
str]:
    with ProcessPoolExecutor(max_workers=workers) as pool:
        digests = pool.map(digest, paths, chunksize=16)
        return dict(zip(paths, digests, strict=True))
```

ProcessPoolExecutor

> Separate interpreters, separate GILs, real parallelism on multiple cores. Same API as the thread pool, so swapping one for the other is a one-word change.

chunksize=16

> Each task has real overhead — arguments are pickled and sent to another process. Batching amortises it. With `chunksize=1` on small tasks, the pool is slower than a plain loop.

The picklability constraint

> Arguments and return values must survive `pickle`. Lambdas, open file handles, and local functions cannot cross the boundary. This is the error you will hit first.

strict=True

> Asserts the two sequences are the same length, so a silent mismatch becomes an exception.

• THE GUARD IS NOT OPTIONAL

```
if __name__ == "__main__":
    main()
```

On Windows and macOS, child processes re-import your module. Without the guard, each child re-runs your top-level code and spawns its own children — a fork bomb.

50.1 Choosing

WORK	USE	WHY
HTTP calls, disk reads, database queries	`ThreadPoolExecutor`	Waiting releases the GIL
Hashing, parsing, compression, numeric loops	`ProcessPoolExecutor`	Real cores, real parallelism
Thousands of concurrent sockets	`asyncio`	One thread, no per-task stack
Anything you have not measured	A plain loop	It is correct, and probably fast enough

```
python3 -m timeit -s 'from mymod import work' 'work()'
time ./tool.py /some/tree >/dev/null
python3 -m cProfile -s cumtime ./tool.py /some/tree | head -30
```

• KEY IDEA

Concurrency comes after correctness. Keep the sequential implementation as the semantic reference and test both against it. "Faster" code that drops errors or reorders output unpredictably is not an optimisation.

• CHALLENGE LEVEL 5

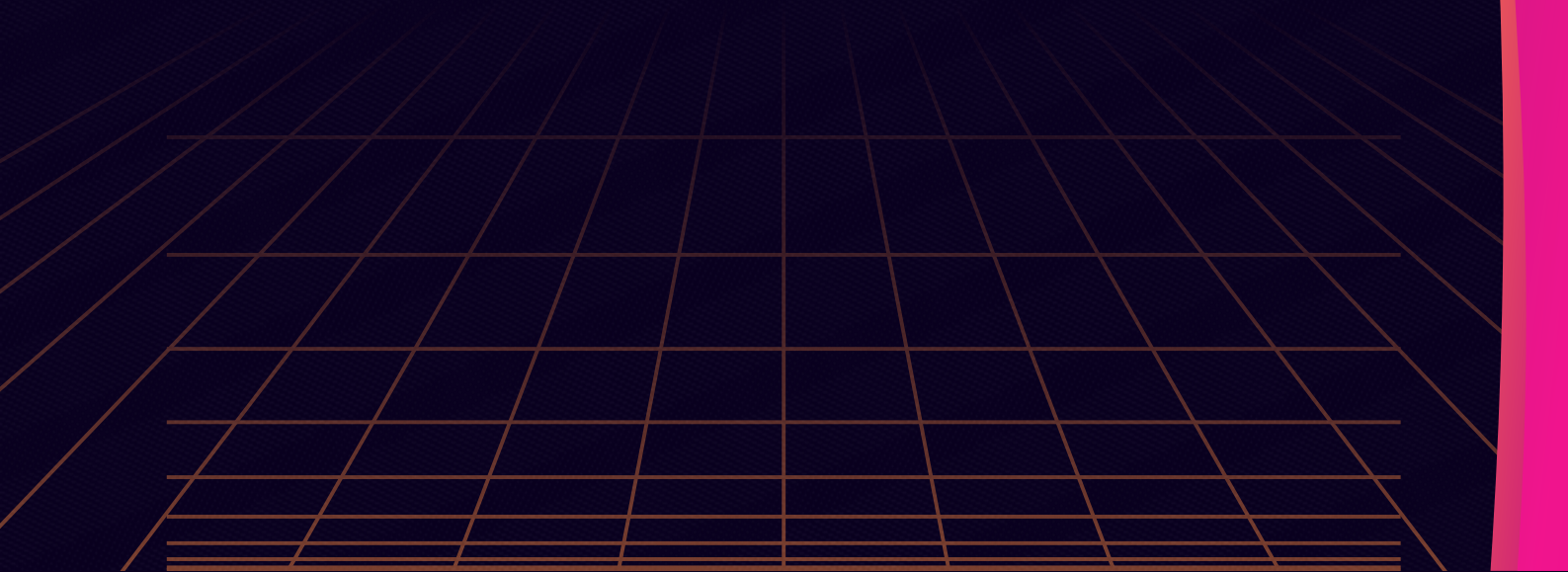
Take the duplicate-finder piece and make it parallel. Measure the sequential version first with `time`. Then produce a threaded version and a process version, and explain the numbers – including why one of them is probably slower than the loop you started with.

Hint: hashing is CPU work, but reading the bytes is I/O, so the answer depends on your disk and file sizes. Both versions must produce *byte-identical* output to the sequential one. If they do not, you have found the real lesson.

PROGRESS – PART IX

- [ ] Ran a child process with a list, a timeout and a checked status
- [ ] Streamed a long-running command's output live
- [ ] Used a thread pool for network work and mapped futures back to inputs
- [ ] Caused a race condition on purpose, then removed the shared state

- [ ] Used a process pool and hit the pickling constraint
- [ ] Profiled before optimising, at least once



PART X

X

CHALLENGES

Rated one to five. No solutions anywhere in this volume — hints only, and the hints point at the decision, not the syntax.

51 HOW THE CHALLENGES WORK

LEVEL	MEANS	IF YOU ARE STUCK
1	Recall. Minutes. Closed-book	Re-read one reference table
2	One function, one clear contract	Write the failing test first
3	Several pieces composed	Draw the data path on paper
4	Real input, real failure modes	Decide the failure policy before coding
5	Design under constraint; no single right answer	Build the boring version, then compare

• RULES OF ENGAGEMENT

1. **Predict, then run.** A prediction you got wrong is worth more than a test that passed.
2. **Type it.** Do not paste.
3. **Write the assertions before the implementation** from level 2 upward.
4. **Run it against hostile input** before you call it done: empty, huge, wrong type, missing file, non-ASCII name.
5. **Then break it on purpose** and read the traceback bottom-up.

A hint tells you which decision matters. It never tells you the call to make.

52 LEVEL 1-2: REPRODUCE AND ISOLATE

• 1.1 LEVEL 1 – PREDICT THE VALUE AND THE TYPE

`7 // -2` · `-7 % 3` · `(1)` · `(1,)` · `{}` · `bool("False")` · `[0] * 3` · `[[0]] * 3` · `"a" in "banana"` · `0.1 + 0.2 == 0.3`

Hint: two of these involve the same object appearing more than once, and one is about binary floating point, not about Python.

• 1.2 LEVEL 1 – NAME THE EXCEPTION

For each, say which exception and at which phase – parse, run, or neither: a missing colon after `if x` · `sys.argv(1)` · `int("12a")` · `{"a": 1}["b"]` · `[1,2][5]` · `open("nope.txt")` · `def f(a, a): pass` · `return size < 1000` where you meant `>`

Hint: exactly one of these produces no error at all, and that is the dangerous one.

• 2.1 LEVEL 2 – PARSE_SIZE

`parse_size("4KiB") -> 4096`. Accepts B, KiB, MiB, GiB, case-insensitive, optional space. Raises `ValueError` with a useful message otherwise.

Hint: the parsing is easy. Decide what `"4"`, `"4 kib"`, `""`, `"-4KiB"` and `"4.5GiB"` mean *before* writing a line, and encode each decision as a test.

• 2.2 LEVEL 2 – INVERT A MAPPING, SAFELY

Write `invert(mapping)` returning keys grouped by value. Then write the version that raises when values are not unique, and decide which one you would actually ship.

Hint: `defaultdict(list)` for the first. For the second, compare lengths – and ask what the caller can do with either outcome.

• 2.3 LEVEL 2 – EXIT STATUSES

Write a `main()` returning `0` for an existing directory, `1` for a path that exists but is not a directory, and `2` for a missing path. Diagnostics to stderr. Prove all three from the shell with `echo $?`.

Hint: `Path.exists()` and `Path.is_dir()` are two questions, not one. A broken symlink answers both surprisingly.

53 LEVEL 3: BUILD SOMETHING WHOLE

• 3.1 LEVEL 3 – TAIL AND FOLLOW

`tail(path, n)` without reading the whole file or seeking. Then `follow(path)`, yielding lines as they are appended.

Hint: `deque(maxlen=n)` is the whole first answer. The second is a design question: what should happen when the file is rotated or truncated underneath you?

• 3.2 LEVEL 3 – @RETRY

`@retry(times=3, delay=0.5, on=(TimeoutError,))`. Preserves identity, re-raises the last exception, never swallows `KeyboardInterrupt`, works on methods.

Hint: a parameterised decorator is three nested `def`s. `functools.wraps` is not optional. Test with a function that fails twice then succeeds, and assert the call count.

• 3.3 LEVEL 3 – A REAL CLI

`dirstat`: total size, file count, the ten largest files, and a count per extension. Flags for `--json`, `--limit`, `--exclude GLOB` (repeatable), and `-v/-vv`. Must be pipeable and must not build a list of every path.

Hint: `heapq.nlargest` keeps ten in memory instead of a million. Decide which stage owns the exclude filter – discovery, collection, or rendering – and you will find only one answer is cheap.

54 LEVEL 4: BREAK IT ON PURPOSE

• HOW TO RUN A BREAK-IT EXERCISE

Take working code. Introduce **one** defect from the list. Run it. Predict the traceback before reading it. Then repair it closed-book. The goal is recognising a failure by its shape, so that the real one costs you thirty seconds instead of an afternoon.

4.1 LEVEL 4 – FIFTEEN DELIBERATE INJURIES

Apply each to a working script, one at a time:

1. Delete a colon from a compound header.
2. Indent one line of a block with a tab.
3. Use a name one line before it is bound.
4. Unpack three values into four names.
5. Assign to an imported module's name.
6. Call a list: `sys.argv(1)`.
7. Pass an argument to a function that takes none.
8. Give a mutable default and call it twice.
9. Rebind `dirnames` instead of slice-assigning.
10. Iterate a path string instead of a directory.
11. Use `re.match` where you meant `re.search`.
12. Consume a generator twice.
13. Swap `or` for `if x is None` where `0` is valid.
14. Catch `Exception` and `pass`.
15. Reverse a comparison operator in a filter.

Hint: five of these raise nothing. Those five are the exercise. For each, write down how you would ever have noticed.

4.2 LEVEL 4 – HOSTILE TRIALS

Run your `dirstat` against: an empty directory · a missing root · a root that is a regular file · filenames with spaces, tabs, newlines and non-ASCII · a broken symlink · a symlink loop · a permission-denied directory · a file deleted mid-scan · a tree too large to hold · `dirstat / | head -5`.

Hint: the last one raises `BrokenPipeError`, and handling it correctly is genuinely subtle – the fix is not a `try` around `print`. Decide which of these ten belong to your program's contract and which are the operating system's business.

• 4.3 LEVEL 4 – THE LOG PIPELINE, END TO END

Stream a gzipped log from stdin or a path, parse with named groups, summarise by level and logger, emit a text report and an optional PNG. All driven by argparse.

Hint: the pieces in Part VIII already fit; the work is the seams. Decide up front what happens when zero lines match, when the chart directory does not exist, and when stdout closes early.

55 LEVEL 5: REDESIGN UNDER CONSTRAINT

• 5.1 LEVEL 5 – THREE INTERNAL MODELS

Implement `dirstat`'s record three ways: a tuple, a dict, and a frozen dataclass. Do not pick a winner in advance. Compare call sites, failure modes, typing, JSON conversion, and the cost of adding a field six months later.

Hint: the honest answer changes with the size of the program. Say at what size it changes, and why.

• 5.2 LEVEL 5 – MAKE IT PARALLEL, AND JUSTIFY IT

Parallelise the duplicate finder. Measure sequential first. Produce a threaded and a process version. Explain the numbers, including why one is probably slower than the loop.

Hint: hashing is CPU, reading is I/O, and which dominates depends on your disk and your file sizes. Output must be byte-identical to the sequential run. If it is not, that is the lesson.

• 5.3 LEVEL 5 – SPLIT THE MODULE

Split your tool into `cli`, `scan`, `model`, `render`. Add a `pyproject.toml` and a console script. No circular imports, tests still green, behaviour unchanged.

Hint: if a cycle appears, the dependency direction is wrong — move the shared model downward. Hiding an import inside a function is a patch over a design error, not a fix.

5.4 LEVEL 5 — THE FINAL ONE

Repair `vaudeville.py` from Appendix A completely, closed-book, in one sitting. Then take it all the way to an installable package with tests. You have every tool in this volume.

Hint: compile, fix one error, compile again. Do not fix five things between runs. The five defects that raise nothing are the ones the compiler will never mention.

56 APPENDIX A — THE SPECIMEN, DISSECTED

The original `vaudeville.py`. It ships next to this document as a real file. It does not parse; that is its purpose.

```
import os
import syslog
import time
import datetime
import getopt
from sys import exit
import sys

for root, dirs, metadata, files in os.walk(directory):
    for file in files:
        datetime = datetime.list_date(b"{%Y_%m_%d_%T:%M:%S}")

if len(sys.argv) > 1:
    directory = sys.argv(datetime[1])
else:
    directory = os.getcwd(datetime)

def usage():
    print("Usage: vaudeville.py directory/file")
    exit(1)

def metadata(verbose: datetime, files,)-> Return:
    yield(function if:

        for

if len(sys.argv) > 1:

def sys.argv(datetime(metadata: datetime, files, verbose[1])): -> Return:

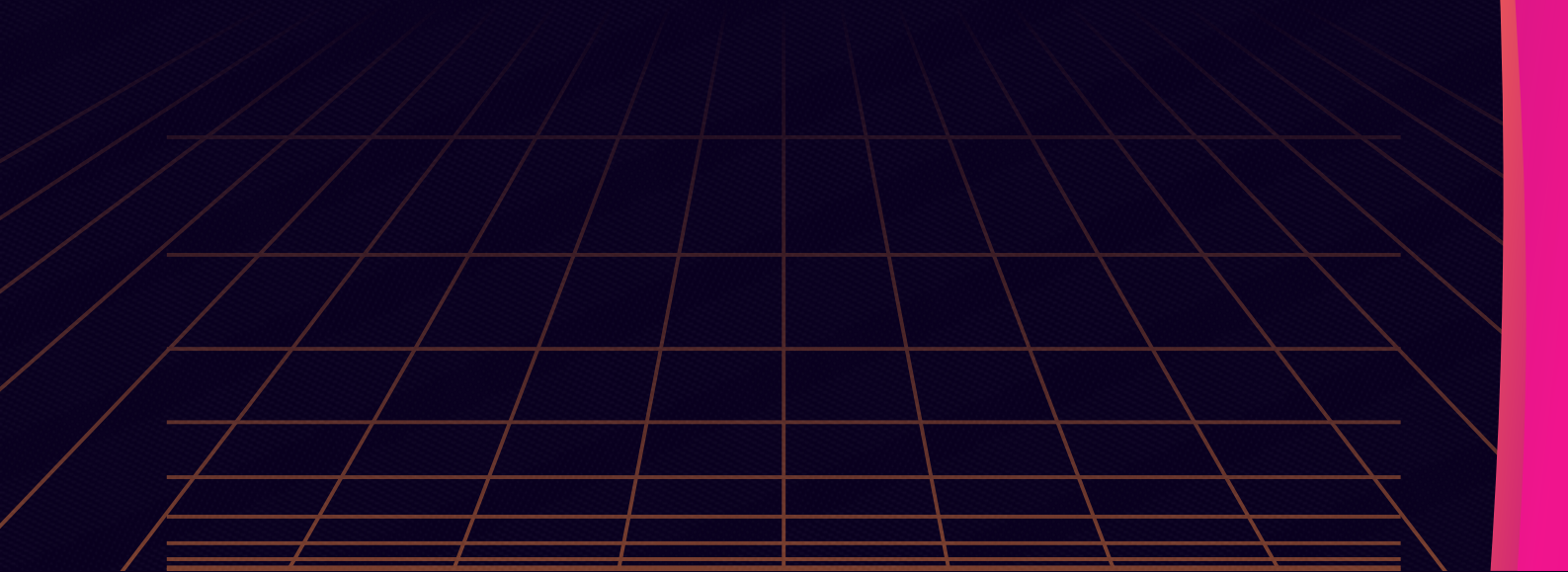
in root, dirs, files in os.walk(directory):
    for file in files:
        os.walk(files: file)

for f in directory:
```

• WHY THIS IS WORTH YOUR TIME

It is not bad code. It is a map of unresolved questions – and every one of them is answered somewhere in this volume:

- What exists before a line executes? – §8
- Which punctuation calls, indexes, annotates, or opens a block? – §7, §9
- What exactly does a library function return? – §9
- When should code be a function, generator, class or loop? – §14, §19, §24
- What is a module, and what does an import bind? – §36, §37
- How does a program receive input and report failure? – §32, §16



56.1 The fifteen defects

#	DEFECT	RAISES	§
1	Four unused imports, and `exit` imported into the namespace	Nothing	36
2	`directory` used before it is bound	`NameError`	8
3	Four names unpacking `os.walk`'s 3-tuple	`ValueError`	8
4	The `datetime` module rebound to a value	Later confusion	8, 36
5	`datetime.list_date` does not exist	`AttributeError`	9
6	`bytes` literal where text is expected	`TypeError`	5, 17
7	`sys.argv(...)` calls a list	`TypeError`	7
8	`os.getcwd(datetime)` takes no arguments	`TypeError`	14
9	Annotations misused; `Return` undefined	`SyntaxError` / `NameError`	21
10	`yield(function if:` – unparseable	`SyntaxError`	25
11	An `if` header with no body	`IndentationError`	11
12	`def sys.argv(...)` – a dotted function name	`SyntaxError`	14
13	A loop starting with `in`	`SyntaxError`	11
14	`os.walk(files: file)` – colon as a keyword argument	`SyntaxError`	7
15	`for f in directory` iterates characters	**Nothing**	8, 40

• THE LESSON, IN ONE LINE

Python must parse the *entire* file before executing any of it. A clean compile proves nothing about defects 1, 4 and 15. Those are found by tests, by reading, and by running the thing against real input.

57 APPENDIX B – ERRORS YOU WILL SEE

WHAT YOU SEE	USUALLY MEANS	§
`SyntaxError: invalid syntax`	Nothing ran. Fix the first one, recompile	56
`IndentationError` / `TabError`	Empty block body, or tabs mixed with spaces	11
`NameError`	Used before bound, or misspelled	8
`UnboundLocalError`	Assignment in the function made the name local	15
`TypeError: not callable`	`()` where you meant `[]`	7
`TypeError: takes N arguments`	Wrong contract – go read the signature	14
`TypeError: unhashable type: 'list'`	A mutable used as a dict key or set member	17
`ValueError: too many values to unpack`	Target count does not match the tuple	24
`AttributeError`	You guessed an API. Use `dir()` instead	9
`KeyError`	Missing key – `dict.get` may be what you want	17
`IndexError`	Off the end of a sequence	17
`FileNotFoundError`	Gone, or never there, or vanished mid-scan	16
`PermissionError`	Expected during traversal. Report, do not crash	16

WHAT YOU SEE	USUALLY MEANS	§
`BrokenPipeError`	Downstream closed — `| head`. Exit quietly	54
`UnicodeDecodeError`	Binary read as text. `errors="replace"` or open `"rb"`	41
`ModuleNotFoundError`	Not installed, wrong venv, or your file shadows it	37
`RecursionError`	No base case, or a symlink loop	25
`StopIteration` leaking	`next()` on an exhausted iterator; pass a default	24
`PicklingError`	A lambda or handle sent to a process pool	50
Prints characters, not files	You iterated a path string. No exception	8
Second call remembers the first	A mutable default argument	14
Loop runs, output is empty	The generator was already consumed	24
Runs, wrong answer, silent	Logic failure. Only tests and thought find these	33

58 APPENDIX C — WHAT COMPETENCE LOOKS LIKE

• THE DEFINITION

Fluency is not remembering every standard-library method. It is being able to:

- [ ] reduce an idea to one working vertical slice;
- [ ] predict how names, objects and control flow interact;
- [ ] interrogate unfamiliar objects rather than inventing their APIs;
- [ ] read a traceback and isolate the failure category;
- [ ] choose between a value, collection, iterator, function, module and class for a reason;
- [ ] keep filesystem, network and process boundaries explicit;

- [ ] make invalid states and failures visible;
- [ ] test the behaviour that matters and skip the behaviour that does not;
- [ ] judge a dependency before adopting it;
- [ ] measure before optimising;
- [ ] revise a design after evidence without treating the first version as sacred.

> *The missing piece was never intelligence or ambition. It was a reliable model of what Python's symbols cause the interpreter to do.*
>
> APPENDIX A — CLOSING

• FINISH

Build that model through repeated contact with real programs, and the language stops looking like punctuation to imitate. It becomes machinery you can reason about.

Ships with this document — `vaudeville.py` , the specimen, unrepaired

Companion volume — Python From the Terminal Outward

Every callout colour means the same thing on every page: pink is the thing to remember, cyan is context, orange is a trick or a memory hook, coral is a trap, violet is a worked example or an annotated piece, amber is a challenge. Set in Retrograde.

